

Each cell in a linked list represents a node. The bucket headers, consisting of pointers to the first cell in a list, are stored in an array. The bucket number returned by $h(op, l, r)$ is an index into this array of bucket headers.

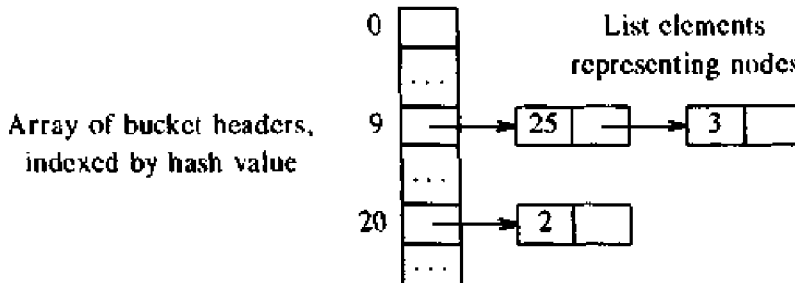


Fig. 5.14. Data structure for searching buckets.

This algorithm can be adapted to apply to nodes that are not allocated sequentially from an array. In many compilers, nodes are allocated as they are needed, to avoid preallocating an array that may hold too many nodes most of the time and not enough nodes some of the time. In this case, we cannot assume that nodes are in sequential storage, so we have to use pointers to refer to nodes. If the hash function can be made to compute the bucket number from a label and pointers to children, then we can use pointers to nodes instead of value numbers. Otherwise, we can number the nodes in any way and use this number as the value number of the node. $\square$

Dags can also be used to represent sets of expressions, since a dag can have more than one root. In Chapters 9 and 10, the computations performed by a sequence of assignment statements will be represented as a dag.

5.3 BOTTOM-UP EVALUATION OF S-ATTRIBUTED DEFINITIONS

Now that we have seen how to use syntax-directed definitions to specify translations, we can begin to study how to implement translators for them. A translator for an arbitrary syntax-directed definition can be difficult to build. However, there are large classes of useful syntax-directed definitions for which it is easy to construct translators. In this section, we examine one such class: the S-attributed definitions, that is, the syntax-directed definitions with only synthesized attributes. The following sections consider the implementation of definitions that have inherited attributes as well.

Synthesized attributes can be evaluated by a bottom-up parser as the input is being parsed. The parser can keep the values of the synthesized attributes associated with the grammar symbols on its stack. Whenever a reduction is made, the values of the new synthesized attributes are computed from the attributes appearing on the stack for the grammar symbols on the right side of the reducing production. This section shows how the parser stack can be extended to hold the values of these synthesized attributes. We shall see in

Section 5.6 that this implementation also supports some inherited attributes.

Only synthesized attributes appear in the syntax-directed definition in Fig. 5.9 for constructing the syntax tree for an expression. The approach of this section can therefore be applied to construct syntax trees during bottom-up parsing. As we shall see in Section 5.5, the translation of expressions during top-down parsing often uses inherited attributes. We therefore defer translation during top-down parsing until after "left-to-right" dependencies are examined in the next section.

Synthesized Attributes on the Parser Stack

A translator for an S-attributed definition can often be implemented with the help of an LR-parser generator, such as the one discussed in Section 4.9. From an S-attributed definition, the parser generator can construct a translator that evaluates attributes as it parses the input.

A bottom-up parser uses a stack to hold information about subtrees that have been parsed. We can use extra fields in the parser stack to hold the values of synthesized attributes. Figure 5.15 shows an example of a parser stack with space for one attribute value. Let us suppose, as in the figure, that the stack is implemented by a pair of arrays *state* and *val*. Each *state* entry is a pointer (or index) to an LR(1) parsing table. (Note that the grammar symbol is implicit in the state and need not be stored in the stack.) It is convenient, however, to refer to the state by the unique grammar symbol that it covers when placed on the parsing stack as described in Section 4.7. If the *i*th *state* symbol is *A*, then *val*[*i*] will hold the value of the attribute associated with the parse tree node corresponding to this *A*.

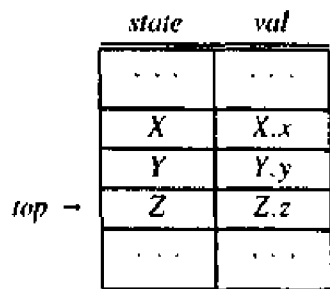


Fig. 5.15. Parser stack with a field for synthesized attributes.

The current top of the stack is indicated by the pointer *top*. We assume that synthesized attributes are evaluated just before each reduction. Suppose the semantic rule $A.a := f(X.x, Y.y, Z.z)$ is associated with the production $A \rightarrow XYZ$. Before XYZ is reduced to A , the value of the attribute $Z.z$ is in *val*[*top*], that of $Y.y$ in *val*[*top* - 1], and that of $X.x$ in *val*[*top* - 2]. If a symbol has no attribute, then the corresponding entry in the *val* array is undefined. After the reduction, *top* is decremented by 2, the state covering A is

put in $state[top]$ (i.e., where X was), and the value of the synthesized attribute $A.a$ is put in $val[top]$.

Example 5.10. Consider again the syntax-directed definition of the desk calculator in Fig. 5.2. The synthesized attributes in the annotated parse tree of Fig. 5.3 can be evaluated by an LR parser during a bottom-up parse of the input line $3*5+4n$. As before, we assume that the lexical analyzer supplies the value of attribute $digit.lexval$, which is the numeric value of each token representing a digit. When the parser shifts a **digit** onto the stack, the token **digit** is placed in $state[top]$ and its attribute value is placed in $val[top]$.

We can use the techniques of Section 4.7 to construct an LR parser for the underlying grammar. To evaluate attributes, we modify the parser to execute the code fragments shown in Fig. 5.16 just before making the appropriate reduction. Note that we can associate attribute evaluation with reductions, because each reduction determines the production to be applied. The code fragments have been obtained from the semantic rules in Fig. 5.2 by replacing each attribute by a position in the val array.

PRODUCTION	CODE FRAGMENT
$L \rightarrow E n$	$print(val[top])$
$E \rightarrow E_1 + T$	$val[ntop] := val[top-2] + val[top]$
$E \rightarrow T$	
$T \rightarrow T_1 * F$	$val[ntop] := val[top-2] \times val[top]$
$T \rightarrow F$	
$F \rightarrow (E)$	$val[ntop] := val[top-1]$
$F \rightarrow \text{digit}$	

Fig. 5.16. Implementation of a desk calculator with an LR parser.

The code fragments do not show how the variables top and $ntop$ are managed. When a production with r symbols on the right side is reduced, the value of $ntop$ is set to $top - r + 1$. After each code fragment is executed, top is set to $ntop$.

Figure 5.17 shows the sequence of moves made by the parser on input $3*5+4n$. The contents of the $state$ and val fields of the parsing stack are shown after each move. We again take the liberty of replacing stack states by their corresponding grammar symbols. We take the further liberty of showing, instead of token **digit**, the actual input digit.

Consider the sequence of events on seeing the input symbol 3. In the first move, the parser shifts the state corresponding to the token **digit** (whose attribute value is 3) onto the stack. (The state is represented by 3 and the value 3 is in the val field.) On the second move, the parser reduces by the production $F \rightarrow \text{digit}$ and implements the semantic rule $F.val := \text{digit.lexval}$. On the third move the parser reduces by $T \rightarrow F$. No code fragment is associated with

INPUT	state	val	PRODUCTION USED
3*5+4 n	-	-	
*5+4 n	3	3	
*5+4 n	F	3	$F \rightarrow \text{digit}$
*5+4 n	T	3	$T \rightarrow F$
5+4 n	T *	3 _	
+4 n	T * 5	3 _ 5	
+4 n	T * F	3 _ 5	$F \rightarrow \text{digit}$
+4 n	T	15	$T \rightarrow T * F$
+4 n	E	15	$E \rightarrow T$
4 n	E +	15 _	
n	E + 4	15 _ 4	
n	E + F	15 _ 4	$F \rightarrow \text{digit}$
n	E + T	15 _ 4	$T \rightarrow F$
n	E	19	$E \rightarrow E + T$
	E n	19 _	
	L	19	$L \rightarrow E n$

Fig. 5.17. Moves made by translator on input 3*5+4 n.

this production, so the *val* array is left unchanged. Note that after each reduction the top of the *val* stack contains the attribute value associated with the left side of the reducing production. $\square$

In the implementation sketched above, code fragments are executed just before a reduction takes place. Reductions provide a "hook" on which actions consisting of arbitrary code fragments can be hung. That is, we can allow the user to associate an action with a production that is executed when a reduction according to that production takes place. Translation schemes considered in the next section provide a notation for interleaving actions with parsing. In Section 5.6, we shall see how a larger class of syntax-directed definitions can be implemented during bottom-up parsing.

5.4 L-ATTRIBUTED DEFINITIONS

When translation takes place during parsing, the order of evaluation of attributes is linked to the order in which nodes of a parse tree are "created" by the parsing method. A natural order that characterizes many top-down and bottom-up translation methods is the one obtained by applying the procedure *dfvisit* in Fig. 5.18 to the root of a parse tree. We call this evaluation order the *depth-first order*. Even if the parse tree is not actually constructed, it is useful to study translation during parsing by considering depth-first evaluation of attributes at the nodes of a parse tree.

```

procedure dfvisit (n: node);
begin
    for each child m of n, from left to right do begin
        evaluate inherited attributes of m;
        dfvisit (m)
    end;
    evaluate synthesized attributes of n
end

```

Fig. 5.18. Depth-first evaluation order for attributes in a parse tree.

We now introduce a class of syntax-directed definitions, called L-attributed definitions, whose attributes can always be evaluated in depth-first order. (The L is for “left,” because attribute information appears to flow from left to right.) Implementation of progressively larger classes of L-attributed definitions is covered in the next three sections of this chapter. L-attributed definitions include all syntax-directed definitions based on LL(1) grammars; Section 5.5 gives a method for implementing such definitions in a single pass using predictive parsing methods. A larger class of L-attributed definitions is implemented in Section 5.6 during bottom-up parsing, by extending the translation methods of Section 5.3. A general method for implementing all L-attributed definitions is outlined in Section 5.7.

L-Attributed Definitions

A syntax-directed definition is *L-attributed* if each inherited attribute of X_j , $1 \leq j \leq n$, on the right side of $A \rightarrow X_1 X_2 \cdots X_n$, depends only on

1. the attributes of the symbols $X_1, X_2, \dots, X_{j-1}$ to the left of X_j in the production and
2. the inherited attributes of A .

Note that every S-attributed definition is L-attributed, because the restrictions (1) and (2) apply only to inherited attributes.

Example 5.11. The syntax-directed definition in Fig. 5.19 is not L-attributed because the inherited attribute $Q.i$ of the grammar symbol Q depends on the attribute $R.s$ of the grammar symbol to its right. Other examples of definitions that are not L-attributed can be found in Sections 5.8 and 5.9. $\square$

Translation Schemes

A translation scheme is a context-free grammar in which attributes are associated with the grammar symbols and semantic actions enclosed between braces $\{\}$ are inserted within the right sides of productions, as in Section 2.3. We

PRODUCTION	SEMANTIC RULES
$A \rightarrow L M$	$L.i := l(A.i)$
	$M.i := m(L.s)$
	$A.s := f(M.s)$
$A \rightarrow Q R$	$R.i := r(A.i)$
	$Q.i := q(R.s)$
	$A.s := f(Q.s)$

Fig. 5.19. A non-L-attributed syntax-directed definition.

shall use translation schemes in this chapter as a useful notation for specifying translation during parsing.

The translation schemes considered in this chapter can have both synthesized and inherited attributes. In the simple translation schemes considered in Chapter 2, the attributes were of string type, one for each symbol, and for every production $A \rightarrow X_1 \cdots X_n$, the semantic rule formed the string for A by concatenating the strings for $X_1, \dots, X_n$, in order, with some optional additional strings in between. We saw that we could perform the translation by simply printing the literal strings in the order they appeared in the semantic rules.

Example 5.12. Here is a simple translation scheme that maps infix expressions with addition and subtraction into corresponding postfix expressions. It is a slight reworking of the translation scheme (2.14) from Chapter 2.

$$\begin{aligned}
 E &\rightarrow T R \\
 R &\rightarrow \text{addop } T \{ \text{print}(\text{addop.lexeme}) \} R_1 \mid \epsilon \\
 T &\rightarrow \text{num } \{ \text{print}(\text{num.val}) \}
 \end{aligned} \tag{5.1}$$

Figure 5.20 shows the parse tree for the input $9-5+2$ with each semantic action attached as the appropriate child of the node corresponding to the left side of their production. In effect, we treat actions as though they are terminal symbols, a viewpoint that is a convenient mnemonic for establishing when the actions are to be executed. We have taken the liberty of showing the actual numbers and additive operator in place of the tokens **num** and **addop**. When performed in depth-first order, the actions in Fig. 5.20 print the output $95-2+$. $\square$

When designing a translation scheme, we must observe some restrictions to ensure that an attribute value is available when an action refers to it. These restrictions, motivated by L-attributed definitions, ensure that an action does not refer to an attribute that has not yet been computed.

The easiest case occurs when only synthesized attributes are needed. For this case, we can construct the translation scheme by creating an action

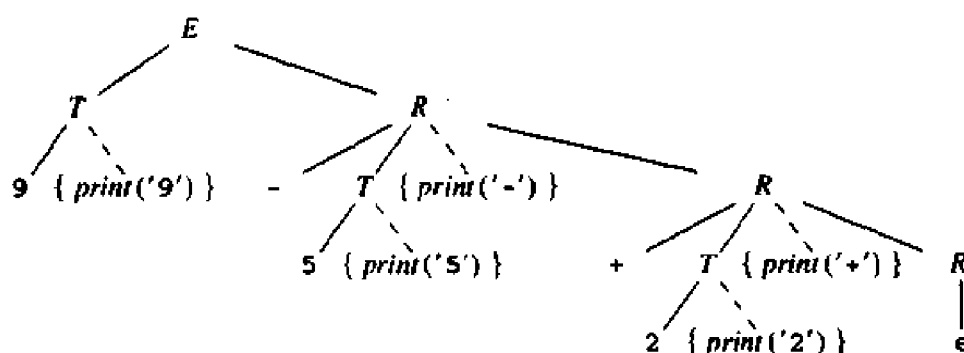


Fig. 5.20. Parse tree for 9-5+2 showing actions.

consisting of an assignment for each semantic rule, and placing this action at the end of the right side of the associated production. For example, the production and semantic rule

PRODUCTION	SEMANTIC RULE
$T \rightarrow T_1 * F$	$T.val := T_1.val \times F.val$

yield the following production and semantic action

$$T \rightarrow T_1 * F \{ T.val := T_1.val \times F.val \}$$

If we have both inherited and synthesized attributes, we must be more careful:

1. An inherited attribute for a symbol on the right side of a production must be computed in an action before that symbol.
2. An action must not refer to a synthesized attribute of a symbol to the right of the action.
3. A synthesized attribute for the nonterminal on the left can only be computed after all attributes it references have been computed. The action computing such attributes can usually be placed at the end of the right side of the production.

In the next two sections, we show how a translation scheme satisfying these three requirements can be implemented by generalizations of top-down and bottom-up parsers.

The following translation scheme does not satisfy the first of these three requirements.

$$\begin{array}{ll} S \rightarrow A_1 A_2 & \{ A_1.in := 1; A_2.in := 2 \} \\ A \rightarrow a & \{ print(A.in) \} \end{array}$$

We find that the inherited attribute $A.in$ in the second production is not defined when an attempt is made to print its value during a depth-first traversal of the parse tree for the input string aa . That is, a depth-first traversal

starts at S and visits the subtrees for A_1 and A_2 before the values of $A_1.in$ and $A_2.in$ are set. If the action defining the values of $A_1.in$ and $A_2.in$ is embedded before the A 's on the right side of $S \rightarrow A_1 A_2$, instead of after, then $A.in$ will be defined each time $print(A.in)$ occurs.

It is always possible to start with an L-attributed syntax-directed definition and construct a translation scheme that satisfies the three requirements above. The next example illustrates this construction. It is based on the mathematics-formatting language EQN, described briefly in Section 1.2. Given the input

E sub 1 .val

EQN places E , 1, and $.val$ in the relative positions and sizes shown in Fig. 5.21. Notice that the subscript 1 is printed in a smaller size and font, and is moved down relative to E and $.val$.

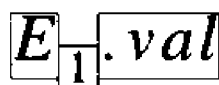


Fig. 5.21. Syntax-directed placement of boxes.

Example 5.13. From the L-attributed definition in Fig. 5.22 we shall construct the translation scheme in Fig. 5.23. In the figures, the nonterminal B (for box) represents a formula. The production $B \rightarrow B B$ represents the juxtaposition of two boxes, and $B \rightarrow B \text{ sub } B$ represents the placement of the second subscript box in a smaller size than the first box in the proper relative position for a subscript.

The inherited attribute ps (for point size) affects the height of a formula. The rule for production $B \rightarrow \text{text}$ causes the normalized height of the text to be multiplied by the point size to get the actual height of the text. The attribute h of text is obtained by table lookup, given the character represented by the token text . When production $B \rightarrow B_1 B_2$ is applied, B_1 and B_2 inherit the point size from B by copy rules. The height of B , represented by the synthesized attribute ht , is the maximum of the heights of B_1 and B_2 .

When production $B \rightarrow B_1 \text{ sub } B_2$ is used, the function *shrink* lowers the point size of B_2 by 30%. The function *disp* allows for the downward displacement of the box B_2 as it computes the height of B . The rules that generate the actual typesetter commands as output are not shown.

The definition in Fig. 5.22 is L-attributed. The only inherited attribute is ps of the nonterminal B . Each semantic rule defines ps only in terms of the inherited attribute of the nonterminal on the left of the production. Hence, the definition is L-attributed.

The translation scheme in Fig. 5.23 is obtained by inserting assignments corresponding to the semantic rules in Fig. 5.22 into the productions,

PRODUCTION	SEMANTIC RULES
$S \rightarrow B$	$B.ps := 10$ $S.ht := B.ht$
$B \rightarrow B_1 B_2$	$B_1.ps := B.ps$ $B_2.ps := B.ps$ $B.ht := \max(B_1.ht, B_2.ht)$
$B \rightarrow B_1 \text{ sub } B_2$	$B_1.ps := B.ps$ $B_2.ps := \text{shrink}(B.ps)$ $B.ht := \text{disp}(B_1.ht, B_2.ht)$
$B \rightarrow \text{text}$	$B.ht := \text{text.h} \times B.ps$

Fig. 5.22. Syntax-directed definition for size and height of boxes.

$$\begin{array}{ll}
 S \rightarrow & \{ B.ps := 10 \} \\
 & B \quad \{ S.ht := B.ht \} \\
 \\
 B \rightarrow & \{ B_1.ps := B.ps \} \\
 & B_1 \quad \{ B_2.ps := B.ps \} \\
 & B_2 \quad \{ B.ht := \max(B_1.ht, B_2.ht) \} \\
 \\
 B \rightarrow & \{ B_1.ps := B.ps \} \\
 & B_1 \\
 & \text{sub} \quad \{ B_2.ps := \text{shrink}(B.ps) \} \\
 & B_2 \quad \{ B.ht := \text{disp}(B_1.ht, B_2.ht) \} \\
 \\
 B \rightarrow \text{text} & \{ B.ht := \text{text.h} \times B.ps \}
 \end{array}$$

Fig. 5.23. Translation scheme constructed from Fig. 5.22.

following the three requirements given above. For readability, each grammar symbol in a production is written on a separate line and actions are shown to the right. Thus,

$$S \rightarrow \{ B.ps := 10 \} \quad B \quad \{ S.ht := B.ht \}$$

is written as

$$\begin{array}{ll}
 S \rightarrow & \{ B.ps := 10 \} \\
 & B \quad \{ S.ht := B.ht \}
 \end{array}$$

Note that actions setting inherited attributes $B_1.ps$ and $B_2.ps$ appear just before B_1 and B_2 on the right sides of productions. $\square$

5.5 TOP-DOWN TRANSLATION

In this section, L-attributed definitions will be implemented during predictive parsing. We work with translation schemes rather than syntax-directed definitions so we can be explicit about the order in which actions and attribute evaluations take place. We also extend the algorithm for left-recursion elimination to translation schemes with synthesized attributes.

Eliminating Left Recursion from a Translation Scheme

Since most arithmetic operators associate to the left, it is natural to use left-recursive grammars for expressions. We now extend the algorithm for eliminating left recursion in Sections 2.4 and 4.3 to allow for attributes when the underlying grammar of a translation scheme is transformed. The transformation applies to translation schemes with synthesized attributes. It allows many of the syntax-directed definitions of Sections 5.1 and 5.2 to be implemented using predictive parsing. The next example motivates the transformation.

Example 5.14. The translation scheme in Fig. 5.24 is transformed below into the translation scheme in Fig. 5.25. The new scheme produces the annotated parse tree of Fig. 5.26 for the expression $9-5+2$. The arrows in the figure suggest a way of determining the value of the expression.

$$\begin{array}{ll}
 E \rightarrow E_1 + T & \{ E.val := E_1.val + T.val \} \\
 E \rightarrow E_1 - T & \{ E.val := E_1.val - T.val \} \\
 E \rightarrow T & \{ E.val := T.val \} \\
 T \rightarrow (E) & \{ T.val := E.val \} \\
 T \rightarrow \text{num} & \{ T.val := \text{num.val} \}
 \end{array}$$

Fig. 5.24. Translation scheme with left-recursive grammar.

In Fig. 5.26, the individual numbers are generated by T , and $T.val$ takes its value from the lexical value of the number, given by attribute num.val . The 9 in the subexpression $9-5$ is generated by the leftmost T , but the minus operator and 5 are generated by the R at the right child of the root. The inherited attribute $R.i$ obtains the value 9 from $T.val$. The subtraction $9-5$ and the passing of the result 4 down to the middle node for R are done by embedding the following action between T and R_1 in $R \rightarrow -TR_1$:

$$\{ R_1.i := R.i - T.val \}$$

A similar action adds 2 to the value of $9-5$, yielding the result $R.i = 6$ at the bottom node for R . The result is needed at the root as the value of $E.val$; the synthesized attribute s for R , not shown in Fig. 5.26, is used to copy the result up to the root. $\square$

For top-down parsing, we can assume that an action is executed at the time that a symbol in the same position would be expanded. Thus, in the second

$$\begin{aligned}
 E &\rightarrow T & \{ R.i &:= T.val \} \\
 R & & \{ E.val &:= R.s \} \\
 R &\rightarrow + \\
 T & & \{ R_1.i &:= R.i + T.val \} \\
 R_1 & & \{ R.s &:= R_1.s \} \\
 R &\rightarrow - \\
 T & & \{ R_1.i &:= R.i - T.val \} \\
 R_1 & & \{ R.s &:= R_1.s \} \\
 R &\rightarrow \epsilon & \{ R.s &:= R.i \} \\
 T &\rightarrow (\\
 E & & \\
) & & \{ T.val &:= E.val \} \\
 T &\rightarrow \text{num} & \{ T.val &:= \text{num.val} \}
 \end{aligned}$$

Fig. 5.25. Transformed translation scheme with right-recursive grammar.

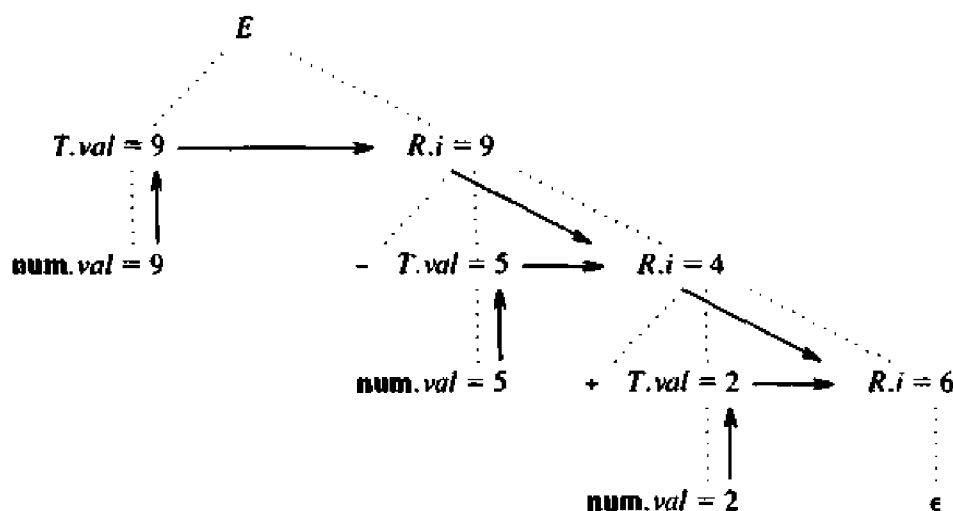


Fig. 5.26. Evaluation of the expression $9-5+2$.

production in Fig. 5.25, the first action (assignment to $R_1.i$) is done after T has been fully expanded to terminals, and the second action is done after R_1 has been fully expanded. As noted in the discussion of L-attributed definitions in Section 5.4, an inherited attribute of a symbol must be computed by an action appearing before the symbol and a synthesized attribute of the non-terminal on the left must be computed after all the attributes it depends on have been computed.

In order to adapt other left-recursive translation schemes for predictive parsing, we shall express the use of attributes $R.i$ and $R.s$ in Fig. 5.25 more abstractly. Suppose we have the following translation scheme

$$\begin{aligned}
 A \rightarrow A_1 Y & \{ A.a := g(A_1.a, Y.y) \} \\
 A \rightarrow X & \{ A.a := f(X.x) \}
 \end{aligned}
 \quad (5.2)$$

Each grammar symbol has a synthesized attribute written using the corresponding lower case letter, and f and g are arbitrary functions. The generalization to additional A -productions and to productions with strings in place of symbols X and Y can be done as in Example 5.15, below.

The algorithm for eliminating left recursion in Section 2.4 constructs the following grammar from (5.2):

$$\begin{aligned}
 A & \rightarrow X R \\
 R & \rightarrow Y R \mid \epsilon
 \end{aligned}
 \quad (5.3)$$

Taking the semantic actions into account, the transformed scheme becomes

$$\begin{aligned}
 A \rightarrow X & \{ R.i := f(X.x) \} \\
 & R \{ A.a := R.s \} \\
 R \rightarrow Y & \{ R_1.i := g(R.i, Y.y) \} \\
 & R_1 \{ R.s := R_1.s \} \\
 R \rightarrow \epsilon & \{ R.s := R.i \}
 \end{aligned}
 \quad (5.4)$$

The transformed scheme uses attributes i and s for R , as in Fig. 5.25. To see why the results of (5.2) and (5.4) are the same, consider the two annotated parse trees in Fig. 5.27. The value of $A.a$ is computed according to (5.2) in Fig. 5.27(a). Figure 5.27(b) contains the computation of $R.i$ down the tree according to (5.4). The value of $R.i$ at the bottom is passed up unchanged as $R.s$, and it becomes the correct value of $A.a$ at the root ($R.s$ is not shown in Fig. 5.27(b)).

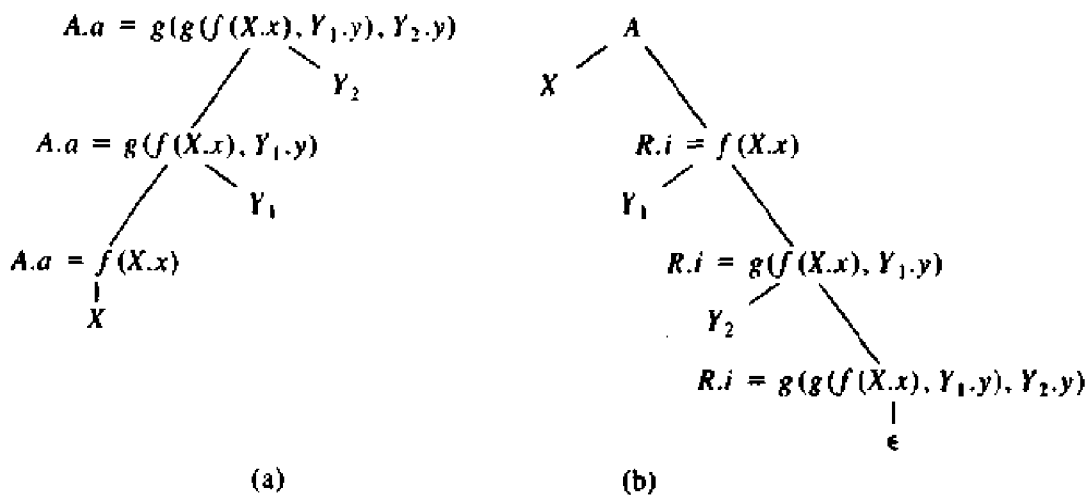


Fig. 5.27. Two ways of computing an attribute value.

Example 5.15. If the syntax-directed definition in Fig. 5.9 for constructing syntax trees is converted into a translation scheme, then the productions and

semantic actions for E become:

$$\begin{aligned} E \rightarrow E_1 + T & \{ E.nptr := mknode(' + ', E_1.nptr, T.nptr) \} \\ E \rightarrow E_1 - T & \{ E.nptr := mknode(' - ', E_1.nptr, T.nptr) \} \\ E \rightarrow T & \{ E.nptr := T.nptr \} \end{aligned}$$

When left recursion is eliminated from this translation scheme, nonterminal E corresponds to A in (5.2) and the strings $+T$ and $-T$ in the first two productions correspond to Y ; nonterminal T in the third production corresponds to X . The transformed translation scheme is shown in Fig. 5.28. The productions and semantic actions for T are similar to those in the original definition in Fig. 5.9.

$$\begin{aligned} E \rightarrow T & \{ R.i := T.nptr \} \\ R & \{ E.nptr := R.s \} \\ \\ R \rightarrow + & \\ T & \{ R_1.i := mknode(' + ', R.i, T.nptr) \} \\ R_1 & \{ R.s := R_1.s \} \\ \\ R \rightarrow - & \\ T & \{ R_1.i := mknode(' - ', R.i, T.nptr) \} \\ R_1 & \{ R.s := R_1.s \} \\ \\ R \rightarrow \epsilon & \{ R.s := R.i \} \\ \\ T \rightarrow (& \\ E & \\) & \{ T.nptr := E.nptr \} \\ \\ T \rightarrow id & \{ T.nptr := mkleaf(id, id.entry) \} \\ \\ T \rightarrow num & \{ T.nptr := mkleaf(num, num.val) \} \end{aligned}$$

Fig. 5.28. Transformed translation scheme for constructing syntax trees.

Figure 5.29 shows how the actions in Fig. 5.28 construct a syntax tree for $a-4+c$. Synthesized attributes are shown to the right of the node for a grammar symbol, and inherited attributes are shown to the left. A leaf in the syntax tree is constructed by actions associated with the productions $T \rightarrow id$ and $T \rightarrow num$, as in Example 5.8. At the leftmost T , attribute $T.nptr$ points to the leaf for a . A pointer to the node for a is inherited as attribute $R.i$ on the right side of $E \rightarrow TR$.

When the production $R \rightarrow -TR_1$ is applied at the right child of the root, $R.i$ points to the node for a , and $T.nptr$ to the node for 4 . The node for $a-4$ is constructed by applying $mknode$ to the minus operator and these pointers.

Finally, when production $R \rightarrow \epsilon$ is applied, $R.i$ points to the root of the entire syntax tree. The entire tree is returned through the s attributes of the nodes for R (not shown in Fig. 5.29) until it becomes the value of $E.nptr$. $\square$

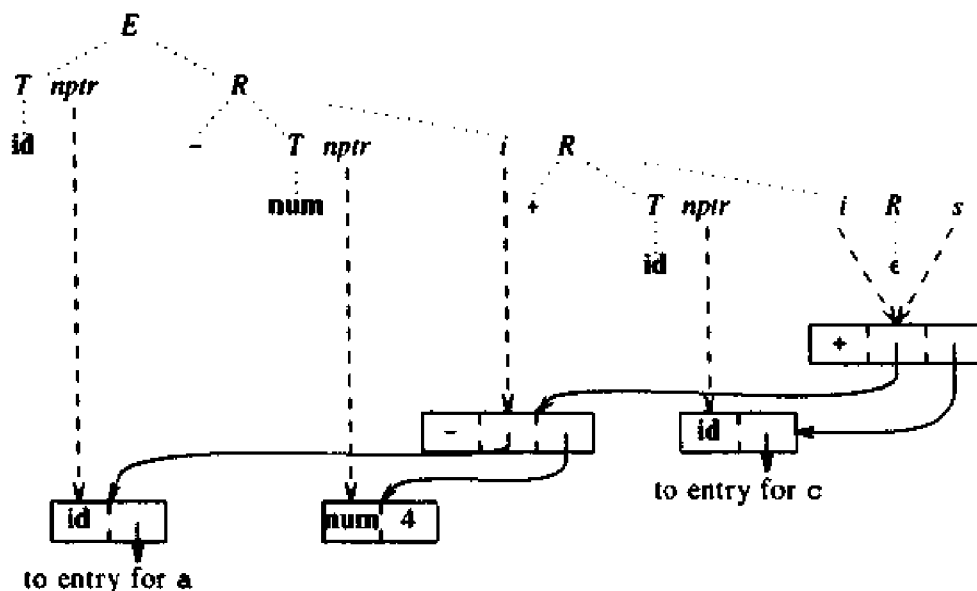


Fig. 5.29. Use of inherited attributes to construct syntax trees.

Design of a Predictive Translator

The next algorithm generalizes the construction of predictive parsers to implement a translation scheme based on a grammar suitable for top-down parsing.

Algorithm 5.2. Construction of a predictive syntax-directed translator.

Input. A syntax-directed translation scheme with an underlying grammar suitable for predictive parsing.

Output. Code for a syntax-directed translator.

Method. The technique is a modification of the predictive-parser construction in Section 2.4.

1. For each nonterminal A , construct a function that has a formal parameter for each inherited attribute of A and that returns the values of the synthesized attributes of A (possibly as a record, as a pointer to a record with a field for each attribute, or using the call-by-reference mechanism for passing parameters, discussed in Section 7.5). For simplicity, we assume that each nonterminal has just one synthesized attribute. The function for A has a local variable for each attribute of each grammar symbol that appears in a production for A .
2. As in Section 2.4, the code for nonterminal A decides what production to use based on the current input symbol.
3. The code associated with each production does the following. We consider the tokens, nonterminals, and actions on the right side of the production from left to right.

- i) For token X with synthesized attribute x , save the value of x in the variable declared for $X.x$. Then generate a call to match token X and advance the input.
- ii) For nonterminal B , generate an assignment $c := B(b_1, b_2, \dots, b_k)$ with a function call on the right side, where $b_1, b_2, \dots, b_k$ are the variables for the inherited attributes of B and c is the variable for the synthesized attribute of B .
- iii) For an action, copy the code into the parser, replacing each reference to an attribute by the variable for that attribute. $\square$

Algorithm 5.2 is extended in Section 5.7 to implement any L-attributed definition, provided a parse tree has already been constructed. In Section 5.8, we consider ways of improving the translators constructed by Algorithm 5.2. For example, it may be possible to eliminate copy statements of the form $x := y$ or to use a single variable to hold the values of several attributes. Some such improvements can also be done automatically using the methods of Chapter 10.

Example 5.16. The grammar in Fig. 5.28 is LL(1), and hence suitable for top-down parsing. From the attributes of the nonterminals in the grammar, we obtain the following types for the arguments and results of the functions for E , R , and T . Since E and T do not have inherited attributes, they have no arguments.

```

function  $E$  :  $\uparrow$  syntax_tree_node;
function  $R(i$  :  $\uparrow$  syntax_tree_node):  $\uparrow$  syntax_tree_node;
function  $T$  :  $\uparrow$  syntax_tree_node;

```

We combine two of the R -productions in Fig. 5.28 to make the translator smaller. The new productions use token **addop** to represent $+$ and $-$:

$$\begin{array}{ll}
 R \rightarrow \text{addop} & \\
 \quad T & \{ R_1.i := \text{mknode}(\text{addop.lexeme}, R.i, T.nptr) \} \\
 \quad R_1 & \{ R.s := R_1.s \} \\
 R \rightarrow \epsilon & \{ R.s := R.i \}
 \end{array} \quad (5.5)$$

The code for R is based on the parsing procedure in Fig. 5.30. If the look-ahead symbol is **addop**, then the production $R \rightarrow \text{addop}TR$ is applied by using the procedure *match* to read the next input token after **addop**, and then calling the procedures for T and R . Otherwise, the procedure does nothing, to mimic the production $R \rightarrow \epsilon$.

The procedure for R in Fig. 5.31 contains code for evaluating attributes. The lexical value *lexval* of the token **addop** is saved in *addoplexeme*, **addop** is matched, T is called, and its result is saved using *nptr*. Variable *il* corresponds to the inherited attribute $R_1.i$, and *sl* to the synthesized attribute $R_1.s$. The **return** statement returns the value of *s* just before control leaves the function. The functions for E and T are constructed similarly. $\square$

```

procedure R;
begin
    if lookahead = addop then begin
        match(addop); T; R
    end
    else begin /* do nothing */
    end
end;

```

Fig. 5.30. Parsing procedure for the productions $R \rightarrow \text{addop } T R \mid \epsilon$.

```

function R(i: ↑ syntax_tree_node): ↑ syntax_tree_node;
    var nptr, il, sl, s: ↑ syntax_tree_node;
        addoplexeme : char;
begin
    if lookahead = addop then begin
        /* production  $R \rightarrow \text{addop } T R$  */
        addoplexeme := lexval;
        match(addop);
        nptr := T;
        il := mknode(addoplexeme, i, nptr);
        sl := R(il);
        s := sl
    end
    else s := i; /* production  $R \rightarrow \epsilon$  */
    return s
end;

```

Fig. 5.31. Recursive-descent construction of syntax trees.

5.6 BOTTOM-UP EVALUATION OF INHERITED ATTRIBUTES

In this section, we present a method to implement L-attributed definitions in the framework of bottom-up parsing. The method is capable of handling all L-attributed definitions considered in the previous section in that it can implement any L-attributed definition based on an LL(1) grammar. It can also implement many (but not all) L-attributed definitions based on LR(1) grammars. The method is a generalization of the bottom-up translation technique introduced in Section 5.3.

Removing Embedding Actions from Translation Schemes

In the bottom-up translation method of Section 5.3, we relied upon all translation actions being at the right end of the production, while in the predictive-

parsing method of Section 5.5 we needed to embed actions at various places within the right side. To begin our discussion of how inherited attributes can be handled bottom up, we introduce a transformation that makes all embedded actions in a translation scheme occur at the right ends of their productions.

The transformation inserts new *marker* nonterminals generating ϵ into the base grammar. We replace each embedded action by a distinct marker nonterminal M and attach the action to the end of the production $M \rightarrow \epsilon$. For example, the translation scheme

$$\begin{aligned} E &\rightarrow T R \\ R &\rightarrow + T \{ \text{print}(' + ') \} R \mid - T \{ \text{print}(' - ') \} R \mid \epsilon \\ T &\rightarrow \text{num} \{ \text{print}(\text{num.val}) \} \end{aligned}$$

is transformed using marker nonterminals M and N into

$$\begin{aligned} E &\rightarrow T R \\ R &\rightarrow + T M R \mid - T N R \mid \epsilon \\ T &\rightarrow \text{num} \{ \text{print}(\text{num.val}) \} \\ M &\rightarrow \epsilon \{ \text{print}(' + ') \} \\ N &\rightarrow \epsilon \{ \text{print}(' - ') \} \end{aligned}$$

The grammars in the two translation schemes accept exactly the same language and, by drawing a parse tree with extra nodes for the actions, we can show that the actions are performed in the same order. Actions in the transformed translation scheme terminate productions, so they can be performed just before the right side is reduced during bottom-up parsing.

Inheriting Attributes on the Parser Stack

A bottom-up parser reduces the right side of production $A \rightarrow XY$ by removing X and Y from the top of the parser stack and replacing them by A . Suppose X has a synthesized attribute $X.s$, which the implementation of Section 5.3 kept along with X on the parser stack.

Since the value of $X.s$ is already on the parser stack before any reductions take place in the subtree below Y , this value can be inherited by Y . That is, if inherited attribute $Y.i$ is defined by the copy rule $Y.i := X.s$, then the value $X.s$ can be used where $Y.i$ is called for. As we shall see, copy rules play an important role in the evaluation of inherited attributes during bottom-up parsing.

Example 5.17. The type of an identifier can be passed by copy rules using inherited attributes as shown in Fig. 5.32 (adapted from Fig. 5.7). We shall first examine the moves made by a bottom-up parser on the input

real p, q, r

Then we show how the value of attribute $T.type$ can be accessed when the productions for L are applied. The translation scheme we wish to implement is

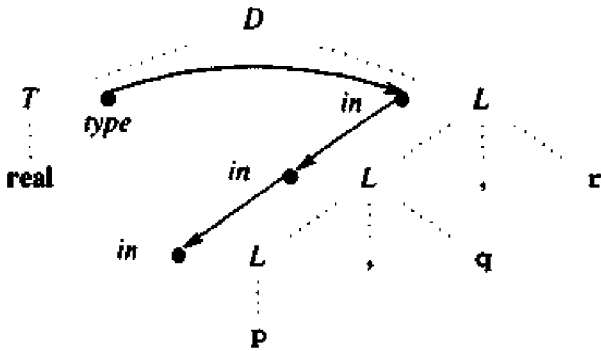


Fig. 5.32. At each node for *L*, *L.in* = *T.type*.

$D \rightarrow T$

L

$T \rightarrow \text{int}$

$T \rightarrow \text{real}$

$L \rightarrow$

L_1, id

$L \rightarrow \text{id}$

$\{ L.in := T.type \}$

$\{ T.type := integer \}$

$\{ T.type := real \}$

$\{ L_1.in := L.in \}$

$\{ addtype(id.entry, L.in) \}$

$\{ addtype(id.entry, L.in) \}$

If we ignore the actions in the above translation scheme, the sequence of moves made by the parser on the input of Fig. 5.32 is as in Fig. 5.33. For clarity, we show the corresponding grammar symbol instead of a stack state and the actual identifier instead of the token *id*.

INPUT	state	PRODUCTION USED
real p,q,r	-	
p,q,r	real	
p,q,r	T	$T \rightarrow \text{real}$
,q,r	T p	
,q,r	T L	$L \rightarrow \text{id}$
q,r	T L ,	
,r	T L , q	
,r	T L	$L \rightarrow L , \text{id}$
r	T L ,	
	T L , r	
	T L	$L \rightarrow L , \text{id}$
	D	$D \rightarrow T L$

Fig. 5.33. Whenever a right side for *L* is reduced, *T* is just below the right side.

Suppose as in Section 5.3 that the parser stack is implemented as a pair of

arrays *state* and *val*. If *state*[*i*] is for grammar symbol *X*, then *val*[*i*] holds a synthesized attribute *X.s*. The contents of the *state* array are shown in Fig. 5.33. Note that every time the right side of a production for *L* is reduced in Fig. 5.33, *T* is in the stack just below the right side. We can use this fact to access the attribute value *T.type*.

The implementation in Fig. 5.34 uses the fact that attribute *T.type* is at a known place in the *val* stack, relative to the top. Let *top* and *ntop* be the indices of the top entry in the stack just before and after a reduction takes place, respectively. From the copy rules defining *L.in*, we know that *T.type* can be used in place of *L.in*.

When the production $L \rightarrow \text{id}$ is applied, *id.entry* is on top of the *val* stack and *T.type* is just below it. Hence, $\text{addtype}(\text{val}[\text{top}], \text{val}[\text{top} - 1])$ is equivalent to $\text{addtype}(\text{id.entry}, T.type)$. Similarly, since the right side of the production $L \rightarrow L, \text{id}$ has three symbols, *T.type* appears in *val*[*top* - 3] when this reduction takes place. The copy rules involving *L.in* are eliminated because the value of *T.type* in the stack is used instead. $\square$

PRODUCTION	CODE FRAGMENT
$D \rightarrow T L ;$	
$T \rightarrow \text{int}$	$\text{val}[\text{ntop}] := \text{integer}$
$T \rightarrow \text{real}$	$\text{val}[\text{ntop}] := \text{real}$
$L \rightarrow L, \text{id}$	$\text{addtype}(\text{val}[\text{top}], \text{val}[\text{top} - 3])$
$L \rightarrow \text{id}$	$\text{addtype}(\text{val}[\text{top}], \text{val}[\text{top} - 1])$

Fig. 5.34. The value of *T.type* is used in place of *L.in*.

Simulating the Evaluation of Inherited Attributes

Reaching into the parser stack for an attribute value works only if the grammar allows the position of the attribute value to be predicted.

Example 5.18. As an instance where we cannot predict the position, consider the following translation scheme:

PRODUCTION	SEMANTIC RULES
$S \rightarrow aAC$	$C.i := A.s$
$S \rightarrow bABC$	$C.i := A.s$
$C \rightarrow c$	$C.s := g(C.i)$

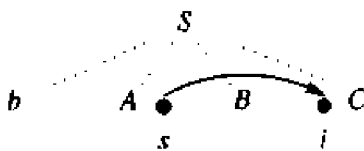
(5.6)

C inherits the synthesized attribute *A.s* by a copy rule. Note that there may or may not be a *B* between *A* and *C* in the stack. When reduction by $C \rightarrow c$ is performed, the value of *C.i* is either in *val*[*top* - 1] or in *val*[*top* - 2], but it is not clear which case applies.

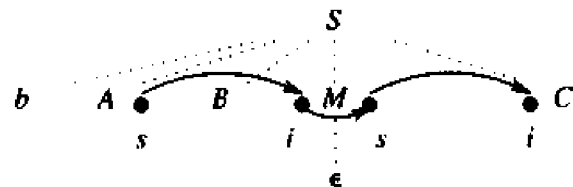
In Fig. 5.35, a fresh marker nonterminal *M* is inserted just before *C* on the

right side of the second production in (5.6). If we are parsing according to production $S \rightarrow bABMC$, then $C.i$ inherits the value of $A.s$ indirectly through $M.i$ and $M.s$. When the production $M \rightarrow \epsilon$ is applied, a copy rule $M.s := M.i$ ensures that the value $M.s = M.i = A.s$ appears just before the part of the stack used for parsing the subtree for C . Thus, the value of $C.i$ can be found in $val[top-1]$ when $C \rightarrow c$ is applied, independent of whether the first or second productions in the following modification of (5.6) are used.

PRODUCTION	SEMANTIC RULES
$S \rightarrow aAC$	$C.i := A.s$
$S \rightarrow bABMC$	$M.i := A.s; C.i := M.s$
$C \rightarrow c$	$C.s := g(C.i)$
$M \rightarrow \epsilon$	$M.s := M.i$



(a) original production



(b) modified dependencies

Fig. 5.35. Copying an attribute value through a marker M .

Marker nonterminals can also be used to simulate semantic rules that are not copy rules. For example, consider

PRODUCTION	SEMANTIC RULES
$S \rightarrow aAC$	$C.i := f(A.s)$

(5.7)

This time the rule defining $C.i$ is not a copy rule, so the value of $C.i$ is not already in the val stack. This problem can also be solved using a marker.

PRODUCTION	SEMANTIC RULES
$S \rightarrow aANC$	$N.i := A.s; C.i := N.s$
$N \rightarrow \epsilon$	$N.s := f(N.i)$

(5.8)

The distinct nonterminal N inherits $A.s$ by a copy rule. Its synthesized attribute $N.s$ is set to $f(A.s)$; then $C.i$ inherits this value using a copy rule. When we reduce by $N \rightarrow \epsilon$, we find the value of $N.i$ in the place for $A.s$, that is, in $val[top-1]$. When we reduce by $S \rightarrow aANC$, the value of $C.i$ is also found in $val[top-1]$, because it is $N.s$. Actually, we do not need $C.i$ at this time; we needed it during the reduction of a terminal string to C , when its value was safely stored on the stack with N .

Example 5.19. Three marker nonterminals L , M , and N are used in Fig. 5.36 to ensure that the value of inherited attribute $B.ps$ appears at a known position in the parser stack while the subtree for B is being reduced. The original

attribute grammar appears in Fig. 5.22 and its relevance to text formatting is explained in Example 5.13.

PRODUCTION	SEMANTIC RULES
$S \rightarrow L B$	$B.ps := L.s$ $S.ht := B.ht$
$L \rightarrow \epsilon$	$L.s := 10$
$B \rightarrow B_1 M B_2$	$B_1.ps := B.ps$ $M.i := B.ps$ $B_2.ps := M.s$ $B.ht := \max(B_1.ht, B_2.ht)$
$B \rightarrow B_1 \text{ sub } N B_2$	$B_1.ps := B.ps$ $N.i := B.ps$ $B_2.ps := N.s$ $B.ht := \text{disp}(B_1.ht, B_2.ht)$
$B \rightarrow \text{text}$	$B.ht := \text{text.h} \times B.ps$
$M \rightarrow \epsilon$	$M.s := M.i$
$N \rightarrow \epsilon$	$N.s := \text{shrink}(N.i)$

Fig. 5.36. All inherited attributes are set by copy rules.

Initialization is done using L . The production for S is $S \rightarrow L B$ in Fig. 5.36, so L will remain on the stack while the subtree below B is reduced. The value 10 of the inherited attribute $B.ps = L.s$ is entered into the parser stack by the rule $L.s := 10$ associated with $L \rightarrow \epsilon$.

Marker M in $B \rightarrow B_1 M B_2$ plays a role similar to that of M in Fig. 5.35; it ensures that the value of $B.ps$ appears just below B_2 in the parser stack. In production $B \rightarrow B_1 \text{ sub } N B_2$, the nonterminal N is used as it is in (5.8). N inherits, via the copy rule $N.i := B.ps$, the attribute value that $B_2.ps$ depends on, and synthesizes the value of $B_2.ps$ by the rule $N.s := \text{shrink}(N.i)$. The consequence, which we leave as an exercise, is that the value of $B.ps$ is always immediately below the right side when we reduce to B .

Code fragments implementing the syntax-directed definition of Fig. 5.36 are shown in Fig. 5.37. All inherited attributes are set by copy rules in Fig. 5.36, so the implementation obtains their values by keeping track of their position in the *val* stack. As in previous examples, *top* and *ntop* give the indices of the top of the stack before and after a reduction, respectively. $\square$

Systematic introduction of markers, as in the modification of (5.6) and (5.7), can make it possible to evaluate L-attributed definitions during LR parsing. Since there is only one production for each marker, a grammar

PRODUCTION	CODE FRAGMENT
$S \rightarrow L B$	$val[top] := val[top]$
$L \rightarrow \epsilon$	$val[top] := 10$
$B \rightarrow B_1 M B_2$	$val[top] := \max(val[top-2], val[top])$
$B \rightarrow B_1 \text{ sub } N B_2$	$val[top] := \text{disp}(val[top-3], val[top])$
$B \rightarrow \text{text}$	$val[top] := val[top] \times val[top-1]$
$M \rightarrow \epsilon$	$val[top] := val[top-1]$
$N \rightarrow \epsilon$	$val[top] := \text{shrink}(val[top-2])$

Fig. 5.37. Implementation of the syntax-directed definition in Fig. 5.36.

remains LL(1) when markers are added. Any LL(1) grammar is also an LR(1) grammar so no parsing conflicts arise when markers are added to an LL(1) grammar. Unfortunately, the same cannot be said of all LR(1) grammars; that is, parsing conflicts can arise if markers are introduced in certain LR(1) grammars.

The ideas of the previous examples can be formalized in the following algorithm.

Algorithm 5.3. Bottom-up parsing and translation with inherited attributes.

Input. An L-attributed definition with an underlying LL(1) grammar.

Output. A parser that computes the values of all attributes on its parsing stack.

Method. Let us assume for simplicity that every nonterminal A has one inherited attribute $A.i$, and every grammar symbol X has a synthesized attribute $X.s$. If X is a terminal, then its synthesized attribute is really the lexical value returned with X by the lexical analyzer; that lexical value appears on the stack, in an array val , as in the previous examples.

For every production $A \rightarrow X_1 \cdots X_n$, introduce n new marker nonterminals, $M_1, \dots, M_n$, and replace the production by $A \rightarrow M_1 X_1 \cdots M_n X_n$.² The synthesized attribute $X_j.s$ will go on the parser stack in the val array entry associated with X_j . The inherited attribute $X_j.i$, if there is one, appears in the same array, but associated with M_j .

An important invariant is that as we parse, the inherited attribute $A.i$, if it exists, is found in the position of the val array immediately below the position for M_1 . As we assume the start symbol has no inherited attribute, there is no problem with the case when the start symbol is A , but even if there were such

² Although inserting M_1 before X_1 simplifies the discussion of marker nonterminals, it has the unfortunate side effect of introducing parsing conflicts into a left-recursive grammar. See Exercise 5.21. As noted below, M_1 can be eliminated.

an inherited attribute, it could be placed below the bottom of the stack. We may prove the invariant by an easy induction on the number of steps in the bottom-up parse, noting the fact that inherited attributes are associated with the marker nonterminals M_j , and that the attribute $X_j.i$ is computed at M_j before we begin the reduction to X_j .

To see that the attributes can be computed as intended during a bottom-up parse, consider two cases. First, if we reduce to a marker nonterminal M_j , we know which production $A \rightarrow M_1X_1 \cdots M_nX_n$ that marker belongs to. We therefore know the positions of any attributes that the inherited attribute $X_j.i$ needs for its computation. $A.i$ is in $\text{val}\{\text{top} - 2j + 2\}$, $X_1.i$ is in $\text{val}\{\text{top} - 2j + 3\}$, $X_1.s$ is in $\text{val}\{\text{top} - 2j + 4\}$, $X_2.i$ is in $\text{val}\{\text{top} - 2j + 5\}$, and so on. Therefore, we may compute $X_j.i$ and store it in $\text{val}\{\text{top} + 1\}$, which becomes the new top of stack after the reduction. Notice how the fact that the grammar is LL(1) is important, or else we might not be sure that we were reducing ϵ to one particular marker nonterminal, and thus could not locate the proper attributes, or even know what formula to apply in general. We ask the reader to take on faith, or derive a proof of the fact that every LL(1) grammar with markers is still LR(1).

The second case occurs when we reduce to a nonmarker symbol, say by production $A \rightarrow M_1X_1 \cdots M_nX_n$. Then, we have only to compute the synthesized attribute $A.s$; note that $A.i$ was already computed, and lives at the position on the stack just below the position into which we insert A itself. The attributes needed to compute $A.s$ are clearly available at known positions on the stack, the positions of the X_j 's, during the reduction.

The following simplifications reduce the number of markers; the second avoids parsing conflicts in left-recursive grammars.

1. If X_j has no inherited attribute, we need not use marker M_j . Of course, the expected positions for attributes on the stack will change if M_j is omitted, but this change can be incorporated easily into the parser.
2. If $X_1.i$ exists, but is computed by a copy rule $X_1.i = A.i$, then we can omit M_1 , since we know by our invariant that $A.i$ will already be located where we want it, just below X_1 on the stack, and this value can therefore serve for $X_1.i$ as well. $\square$

Replacing Inherited by Synthesized Attributes

It is sometimes possible to avoid the use of inherited attributes by changing the underlying grammar. For example, a declaration in Pascal can consist of a list of identifiers followed by a type, e.g., $m, n : \text{integer}$. A grammar for such declarations may include productions of the form

$$\begin{aligned} D &\rightarrow L : T \\ T &\rightarrow \text{integer} \mid \text{char} \\ L &\rightarrow L , \text{id} \mid \text{id} \end{aligned}$$

Since identifiers are generated by L but the type is not in the subtree for L , we cannot associate the type with an identifier using synthesized attributes alone. In fact, if nonterminal L inherits a type from T to its right in the first production, we get a syntax-directed definition that is not L -attributed, so translations based on it cannot be done during parsing.

A solution to this problem is to restructure the grammar to include the type as the last element of the list of identifiers:

$$\begin{aligned} D &\rightarrow \text{id } L \\ L &\rightarrow , \text{id } L \mid : T \\ T &\rightarrow \text{integer} \mid \text{char} \end{aligned}$$

Now, the type can be carried along as a synthesized attribute $L.type$. As each identifier is generated by L , its type can be entered into the symbol table.

A Difficult Syntax-Directed Definition

Algorithm 5.3, for implementing inherited attributes during bottom-up parsing, extends to some, but not all, LR grammars. The L -attributed definition in Fig. 5.38 is based on a simple LR(1) grammar, but it cannot be implemented, as is, during LR parsing. Nonterminal L in $L \rightarrow \epsilon$ inherits the count of the number of 1's generated by S . Since the production $L \rightarrow \epsilon$ is the first that a bottom-up parser would reduce by, the translator at that time cannot know the number of 1's in the input.

PRODUCTION	SEMANTIC RULES
$S \rightarrow L$	$L.count := 0$
$L \rightarrow L_1 1$	$L_1.count := L.count + 1$
$L \rightarrow \epsilon$	$print(L.count)$

Fig. 5.38. Difficult syntax-directed definition.

5.7 RECURSIVE EVALUATORS

Recursive functions that evaluate attributes as they traverse a parse tree can be constructed from a syntax-directed definition using a generalization of the techniques for predictive translation in Section 5.5. Such functions allow us to implement syntax-directed definitions that cannot be implemented simultaneously with parsing. In this section, we associate a single translation function with each nonterminal. The function visits the children of a node for the nonterminal in some order determined by the production at the node; it is not necessary that the children be visited in a left-to-right order. In Section 5.10, we shall see how the effect of translation during more than one pass can be achieved by associating multiple procedures with nonterminals.

Left-to Right Traversals

In Algorithm 5.2, we showed how an L-attributed definition based on an LL(1) grammar can be implemented by constructing a recursive function that parses and translates each nonterminal. All L-attributed syntax-directed definitions can be implemented if a similar recursive function is invoked on a node for that nonterminal in a previously constructed parse tree. By looking at the production at the node, the function can determine what its children are. The function for a nonterminal A takes a node and the values of the inherited attributes for A as arguments, and returns the values of the synthesized attributes for A as results.

The details of the construction are exactly as in Algorithm 5.2, except for step 2 where the function for a nonterminal decides what production to use based on the current input symbol. The function here employs a case statement to determine the production used at a node. We give an example to illustrate the method.

Example 5.20. Consider the syntax-directed definition for determining the size and height of formulas in Fig. 5.22. The nonterminal B has an inherited attribute ps and a synthesized attribute ht . Using Algorithm 5.2, modified as mentioned above, we construct the function for B shown in Fig. 5.39.

Function B takes a node n and a value corresponding to $B.ps$ at the node as arguments, and returns a value corresponding to $B.ht$ at node n . The function has a case for each production with B on the left. The code corresponding to each production simulates the semantic rules associated with the production. The order in which the rules are applied must be such that inherited attributes of a nonterminal are computed before the function for the nonterminal is called.

In the code corresponding to the production $B \rightarrow B \text{ sub } B$, variables ps , $ps1$, and $ps2$ hold the values of the inherited attributes $B.ps$, $B_1.ps$, and $B_2.ps$. Similarly ht , $ht1$, and $ht2$ hold the values of $B.ht$, $B_1.ht$, and $B_2.ht$. We use the function $child(m, i)$ to refer to the i th child of node m . Since B_2 is the label of the third child of node n , the value of $B_2.ht$ is determined by the function call $B(child(n, 3), ps2)$. $\square$

Other Traversals

Once an explicit parse tree is available, we are free to visit the children of a node in any order. Consider the non-L-attributed definition of Example 5.21. In a translation specified by this definition, the children of a node for one production need to be visited from left to right, while the children of a node for the other production need to be visited from right to left.

This abstract example illustrates the power of using mutually recursive functions for evaluating the attributes at the nodes of a parse tree. The functions need not depend on the order in which the parse tree nodes are created. The main consideration for evaluation during a traversal is that the inherited attributes at a node be computed before the node is first visited and that the

```

function  $B(n, ps)$ ;
    var  $ps1, ps2, ht1, ht2$ ;
begin
    case production at node  $n$  of
        ' $B \rightarrow B_1 B_2$ ':
             $ps1 := ps$ ;
             $ht1 := B(\text{child}(n, 1), ps1)$ ;
             $ps2 := ps$ ;
             $ht2 := B(\text{child}(n, 2), ps2)$ ;
            return  $\max(ht1, ht2)$ ;
        ' $B \rightarrow B_1 \text{ sub } B_2$ ':
             $ps1 := ps$ ;
             $ht1 := B(\text{child}(n, 1), ps1)$ ;
             $ps2 := \text{shrink}(ps)$ ;
             $ht2 := B(\text{child}(n, 3), ps2)$ ;
            return  $\text{disp}(ht1, ht2)$ ;
        ' $B \rightarrow \text{text}$ ':
            return  $ps \times \text{text}.h$ ;
    default:
         $\text{error}$ 
    end
end;

```

Fig. 5.39. Function for nonterminal B in Fig. 5.22.

synthesized attributes be computed before we leave the node for the last time.

Example 5.21. Each of the nonterminals in Fig. 5.40 has an inherited attribute i and a synthesized attribute s . The dependency graphs for the two productions are also shown. The rules associated with $A \rightarrow LM$ set up left-to-right dependencies and the rules associated with $A \rightarrow QR$ set up right-to-left dependencies.

The function for nonterminal A is shown in Fig. 5.41; we assume that functions for L , M , Q , and R can be constructed. Variables in Fig. 5.41 are named after the nonterminal and its attribute; e.g., li and ls are the variables corresponding to $L.i$ and $L.s$.

The code corresponding to the production $A \rightarrow LM$ is constructed as in Example 5.20. That is, we determine the inherited attribute of L , call the function for L to determine the synthesized attribute of L , and repeat the process for M . The code corresponding to $A \rightarrow QR$ visits the subtree for R before it visits the subtree for Q . Otherwise, the code for the two productions is very similar. $\square$

PRODUCTION	SEMANTIC RULES
$A \rightarrow L M$	$L.i := l(A.i)$ $M.i := m(L.s)$ $A.s := f(M.s)$
$A \rightarrow Q R$	$R.i := r(A.i)$ $Q.i := q(R.s)$ $A.s := f(Q.s)$



Fig. 5.40. Productions and semantic rules for nonterminal A .

```

function A(n, ai);
begin
  case production at node n of
    'A → L M':      /* left-to-right order */
      li := l(ai);
      ls := L(child(n, 1), li);
      mi := m(ls);
      ms := M(child(n, 2), mi);
      return f(ms);
    'A → Q R':      /* right-to-left order */
      ri := r(ai);
      rs := R(child(n, 2), ri);
      qi := q(rs);
      qs := Q(child(n, 1), qi);
      return f(qs);
  default:
    error
  end
end;

```

Fig. 5.41. Dependencies in Fig. 5.40 determine the order children are visited in.

5.8 SPACE FOR ATTRIBUTE VALUES AT COMPILE TIME

In this section we consider the compile-time assignment of space for attribute values. We shall use information from the dependency graph for a parse tree, so the approach of this section is suitable for parse-tree methods that determine the evaluation order from the dependency graph. In the next section we consider the case in which the evaluation order can be predicted in advance, so we can decide on the space for attributes once and for all when the compiler is constructed.

Given a not necessarily depth-first order of evaluation for attributes, the *lifetime* of an attribute begins when the attribute is first computed and ends when all attributes that depend on it have been computed. We can conserve space by holding an attribute value only during its lifetime.

In order to emphasize that the techniques in this section apply to any evaluation order, we shall consider the following syntax-directed definition that is not L-attributed, for passing type information to identifiers in a declaration.

Example 5.22. The syntax-directed definition in Fig. 5.42 is an extension of that in Fig. 5.4 to allow declarations of the form

real c[12][31]; (5.9)

int x[3], y[5]; (5.10)

A parse tree for (5.10) is shown by the dotted lines in Fig. 5.43(a). The numbers at the nodes are discussed in the next example. As in Example 5.3, the type obtained from T is inherited by L and passed down towards the identifiers in the declaration. An edge from $T.type$ to $L.in$ shows that $L.in$ depends on $T.type$. The syntax-directed definition in Fig. 5.42 is not L-attributed because $L.in$ depends on $num.val$ and num is to the right of L in $L \rightarrow L_1 [num]$. $\square$

Assigning Space for Attributes at Compile Time

Suppose we are given a sequence of registers to hold attribute values. For convenience, we assume that each register can hold any attribute value. If attributes represent different types, then we can form groups of attributes that take the same amount of storage and consider each group separately. We rely on information about the lifetimes of attributes to determine the registers into which they are evaluated.

Example 5.23. Suppose that attributes are evaluated in the order given by the node numbers in the dependency graph of Fig. 5.43,³ constructed in the last example. The lifetime of each node begins when its attribute is evaluated and

³ The dependency graph in Fig. 5.43 does not show nodes corresponding to the semantic rule $addtype(id.entry, l.in)$ because no space is allocated for dummy attributes. Note, however, that this semantic rule must not be evaluated until after the value of $l.in$ is available. An algorithm to determine this fact must work with a dependency graph containing nodes for this semantic rule.

PRODUCTION	RULES
$D \rightarrow T L$	$L.in := T.type$
$T \rightarrow \text{int}$	$T.type := \text{integer}$
$T \rightarrow \text{real}$	$T.type := \text{real}$
$L \rightarrow L_1 , I$	$L_1.in := L.in$ $I.in := L.in$
$L \rightarrow I$	$I.in := L.in$
$I \rightarrow I_1 [\text{num}]$	$I_1.in := \text{array}(\text{num.val}, I.in)$
$I \rightarrow \text{id}$	$\text{addtype}(\text{id.entry}, I.in)$

Fig. 5.42. Passing the type to identifiers in a declaration.

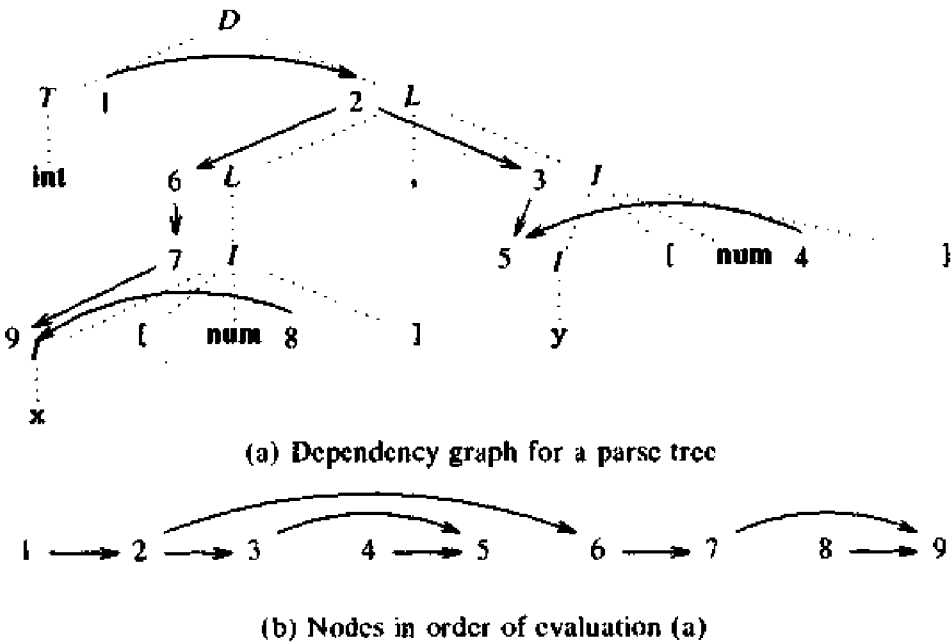


Fig. 5.43. Determining lifetimes of attribute values.

ends when its attribute is used for the last time. For example, the lifetime of node 1 ends when 2 is evaluated because 2 is the only node that depends on 1. The lifetime of 2 ends when 6 is evaluated. □

A method for evaluating attributes that uses as few registers as possible is given in Fig. 5.44. We consider the nodes of dependency graph *D* for a parse

tree in the order they are to be evaluated. Initially, we have a pool of registers $r_1, r_2, \dots$. If attribute b is defined by the semantic rule $b := f(c_1, c_2, \dots, c_k)$, then the lifetime of one or more of $c_1, c_2, \dots, c_k$ might end with the evaluation of b ; registers holding such attributes are returned after b is evaluated. Whenever possible, b is evaluated into a register that held one of $c_1, c_2, \dots, c_k$.

```

for each node  $m$  in  $m_1, m_2, \dots, m_N$  do begin
  for each node  $n$  whose lifetime ends with the evaluation of  $m$  do
    mark  $n$ 's register;
  if some register  $r$  is marked then begin
    unmark  $r$ ;
    evaluate  $m$  into register  $r$ ;
    return marked registers to the pool
  end
  else /* no registers were marked */
    evaluate  $m$  into a register from the pool;
  /* actions using the value of  $m$  can be inserted here */
  if the lifetime of  $m$  has ended then
    return  $m$ 's register to the pool
end

```

Fig. 5.44. Assigning attribute values to registers.

Registers used during an evaluation of the dependency graph of Fig. 5.43 are shown in Fig. 5.45. We start by evaluating node 1 to register r_1 . The lifetime of node 1 ends when 2 is evaluated, so 2 is evaluated into r_1 . Node 3 gets a fresh register r_2 , because node 6 will need the value of 2.

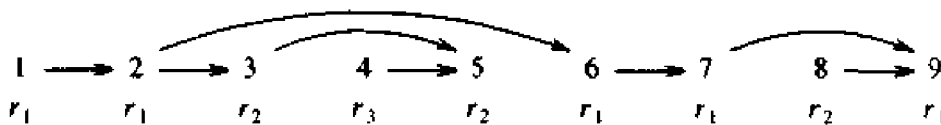


Fig. 5.45. Registers used for attribute values in Fig. 5.43.

Avoiding Copies

We can improve the method of Fig. 5.44 by treating copy rules as a special case. A copy rule has the form $b := c$, so if the value of c is in register r , then the value of b already appears in register r . The number of attributes defined by copy rules can be significant, so we wish to avoid making explicit copies.

A set of nodes having the same value forms an equivalence class. The method of Fig. 5.44 can be modified as follows to hold the value of an equivalence class in a register. When node m is considered, we first check if it is defined by a copy rule. If it is, then its value must already be in a register, and m joins the equivalence class with values in that register. Furthermore, a register is returned to the pool only at the end of the lifetimes of all nodes with values in the register.

Example 5.24. The dependency graph in Fig. 5.43 is redrawn in Fig. 5.46, with an equal sign before each node defined by a copy rule. From the syntax-directed definition in Fig. 5.42, we find that the type determined at node 1 is copied to each element in the list of identifiers, resulting in nodes 2, 3, 6, and 7 of Fig. 5.43 being copies of 1.

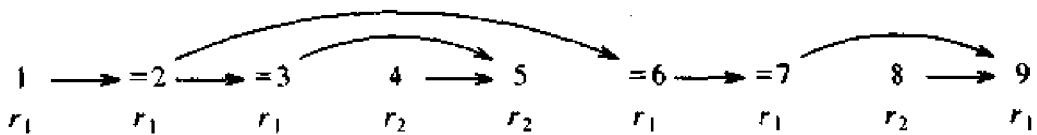


Fig. 5.46. Registers used, taking copy rules into account.

Since 2 and 3 are copies of 1, their values are taken from register r_1 in Fig. 5.46. Note that the lifetime of 3 ends when 5 is evaluated, but register r_1 holding the value of 3 is not returned to the pool because the lifetime of 2 in its equivalence class has not ended.

The following code shows how the declaration (5.10) of Example 5.22 might be processed by a compiler:

```

r1 := integer;          /* evaluates nodes 1, 2, 3, 6, 7 */
r2 := 5;                /* evaluates node 4 */
r2 := array(r2, r1);    /* type of y */
addtype(y, r2);
r2 := 3;                /* evaluates node 8 */
r2 := array(r2, r1);    /* type of x */
addtype(x, r2);
  
```

In the above, x and y point to the symbol-table entries for x and y , and procedure *addtype* must be called at the appropriate times to add the types of x and y to their symbol-table entries. □

5.9 ASSIGNING SPACE AT COMPILER-CONSTRUCTION TIME

Although it is possible to hold all attribute values on a single stack during a traversal, we can sometimes avoid making copies by using multiple stacks. In general, if dependencies between attributes make it inconvenient to place certain attribute values on a stack, we can save them at nodes in an explicitly

constructed syntax tree.

We have already seen the use of a stack to hold attribute values during bottom-up parsing in Sections 5.3 and 5.6. A stack is also used implicitly by a recursive-descent parser to keep track of procedure calls; this issue will be discussed in Chapter 7.

The use of a stack can be combined with other techniques for saving space. The print actions used extensively in the translation schemes in Chapter 2 emit string-valued attributes to an output file whenever possible. While constructing syntax trees in Section 5.2, we passed pointers to nodes instead of complete subtrees. In general, rather than passing large objects, we can save space by passing pointers to them. These techniques will be applied in Examples 5.27 and 5.28.

Predicting Lifetimes from the Grammar

When the evaluation order for attributes is obtained from a particular traversal of the parse tree, we can predict the lifetimes of attributes at compiler-construction time. For example, suppose children are visited from left to right during a depth-first traversal, as in Section 5.4. Starting at a node for production $A \rightarrow BC$, the subtree for B is visited, the subtree for C is visited, and then we return to the node for A . The parent of A cannot refer to the attributes of B and C , so their lifetimes must end when we return to A . Note that these observations are based on the production $A \rightarrow BC$ and the order in which the nodes for these nonterminals are visited. We do not need to know about the subtrees at B and C .

With any evaluation order, if the lifetime of attribute c is contained in that of b , then the value of c can be held in a stack above the value of b . Here b and c do not have to be attributes of the same nonterminal. For the production $A \rightarrow BC$, we can use a stack during a depth-first traversal in the following way.

Start at the node for A with the inherited attributes of A already on the stack. Then evaluate and push the values of the inherited attributes of B . These attributes remain on the stack as we traverse the subtree of B , returning with the synthesized attributes of B above them. This process is repeated with C ; that is, we push its inherited attributes, traverse its subtree and return with its synthesized attributes on top. Writing $I(X)$ and $S(X)$ for the inherited and synthesized attributes of X , respectively, the stack now contains

$$I(A), I(B), S(B), I(C), S(C) \quad (5.11)$$

All of the attribute values needed to compute the synthesized attributes of A are now on the stack, so we can return to A with the stack containing

$$I(A), S(A)$$

Notice that the number (and presumably the size) of inherited and synthesized attributes of a grammar symbol is fixed. Thus, at each step of the above process we know how far down into the stack we have to reach to find

an attribute.

Example 5.25. Suppose that attribute values for the typesetting translation of Fig. 5.22 are held in a stack as discussed above. Starting at a node for production $B \rightarrow B_1 B_2$ with $B.ps$ on top of the stack, the stack contents before and after visiting a node are shown in Fig. 5.47 to the left and right of the node, respectively. As usual stacks grow downwards.

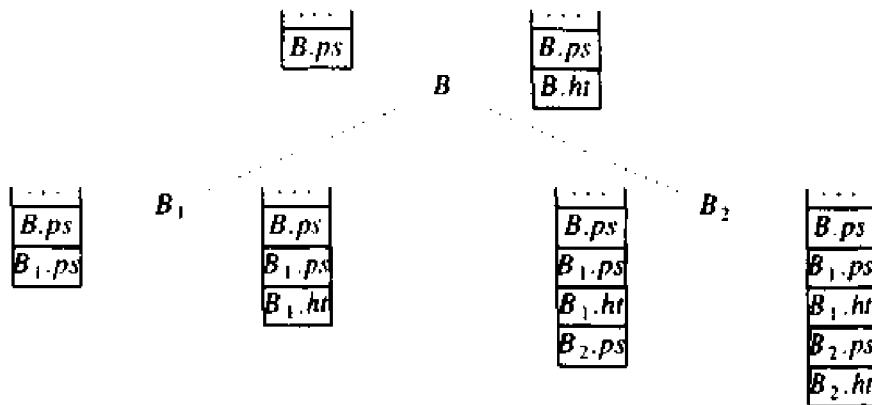


Fig. 5.47. Stack contents before and after visiting a node.

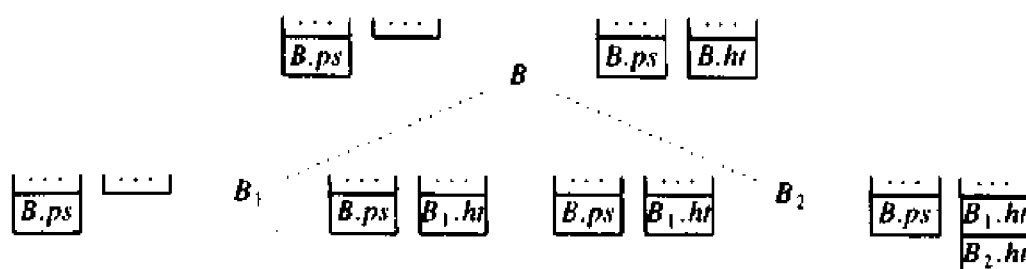
Note that just before a node for nonterminal B is visited for the first time, its ps attribute is on top of the stack. Just after the last visit, i.e., when the traversal moves up from that node, its ht and ps attributes are in the top two positions of the stack. $\square$

When an attribute b is defined by a copy rule $b := c$ and the value of c is on top of the stack of attribute values, it may not be necessary to push a copy of c onto the stack. There may be more opportunities for eliminating copy rules if more than one stack is used to hold attribute values. In the next example, we use separate stacks for synthesized and inherited attributes. A comparison with Example 5.25 shows that more copy rules can be eliminated if separate stacks are used.

Example 5.26. With the syntax-directed definition of Fig. 5.22, suppose we use separate stacks for the inherited attribute ps and the synthesized attribute ht . We maintain the stacks so that $B.ps$ is on top of the ps stack just before B is first visited and just after B is last visited. $B.ht$ will be on top of the ht stack just after B is visited.

With separate stacks we can take advantage of both the copy rules $B_1.ps := B.ps$ and $B_2.ps := B.ps$ associated with $B \rightarrow B_1 B_2$. As shown in Fig. 5.48, we do not need to push $B_1.ps$ because its value is already on top of the stack as $B.ps$.

A translation scheme based on the syntax-directed definition of Fig. 5.22 is shown in Fig. 5.49. Operation $push(v, s)$ pushes the value v onto stack s and

Fig. 5.48. Using separate stacks for attributes *ps* and *ht*

pop(s) pops the value on top of stack *s*. We use *top(s)* to refer to the top element of stack *s*. □

```

S →      { push(10, ps) }
  B

B → B1
   B2 { h2 := top(ht); pop(ht);
        h1 := top(ht); pop(ht);
        push(max(h1, h2), ht) }

B → B1
   sub { push(shrink(top(ps)), ps) }
   B2 { pop(ps);
        h2 := top(ht); pop(ht);
        h1 := top(ht); pop(ht);
        push(disp(h1, h2), ht) }

B → text { push(text.h × top(ps), ht) }

```

Fig. 5.49. Translation scheme maintaining stacks *ps* and *ht*.

The next example combines the use of a stack for attribute values with actions to emit code.

Example 5.27. Here, we consider techniques for implementing a syntax-directed definition specifying the generation of intermediate code. The value of a boolean expression *E* and *F* is false if *E* is false. In C, subexpression *F* must not be evaluated if *E* is false. The evaluation of such boolean expressions is considered in Section 8.4.

Boolean expressions in the syntax-directed definition in Fig. 5.50 are constructed from identifiers and the **and** operator. Each expression *E* inherits two labels *E.true* and *E.false* marking the points control must jump to if *E* is true and false, respectively.

PRODUCTION	SEMANTIC RULES
$E \rightarrow E_1 \text{ and } E_2$	$E_1.true := newlabel$ $E_1.false := E.false$ $E_2.true := E.true$ $E_2.false := E.false$ $E.code := E_1.code \parallel gen('label' E_1.true) \parallel E_2.code$
$E \rightarrow id$	$E.code := gen('if id.place goto E.true) \parallel$ $gen('goto' E.false)$

Fig. 5.50. Short-circuit evaluation of boolean expressions.

Suppose $E \rightarrow E_1 \text{ and } E_2$. If E_1 evaluates to false, then control flows to the inherited label $E.false$; otherwise, E_1 evaluates to true so control flows to the code for evaluating E_2 . A new label generated by function *newlabel* marks the beginning of the code for E_2 . Individual instructions are formed using *gen*. For further discussion of the relevance of Fig. 5.50 to intermediate code generation see Section 8.4.

The syntax-directed definition in Fig. 5.50 is L-attributed, so we can construct a translation scheme for it. The translation scheme in Fig. 5.51 uses a procedure *emit* to generate and emit instructions incrementally. Also shown in the figure are actions for setting the values of the inherited attributes, inserted before the appropriate grammar symbol as discussed in Section 5.4.

```

E →      {  $E_1.true := newlabel;$ 
            $E_1.false := E.false$  }
      E1
      and {  $emit('label' E_1.true);$ 
            $E_2.true := E.true;$ 
            $E_2.false := E.false$  }
      E2

E → id   {  $emit('if id.place goto E.true);$ 
            $emit('goto' E.false) \}$ 

```

Fig. 5.51. Emitting code for boolean expressions.

The translation scheme in Fig. 5.52 goes further; it uses separate stacks to hold the values of the inherited attributes $E.true$ and $E.false$. As in Example 5.26, copy rules have no effect on the stacks. To implement the rule $E_1.true := newlabel$, a new label is pushed onto the true stack before E_1 is visited. The lifetime of this label ends with the action $emit('label' top(true))$, corresponding to $emit('label' E_1.true)$, so the true stack is popped after the action. The *false* stack does not change in this example, but it is needed when the *or* operator is allowed in addition to the *and* operator. □

$$\begin{array}{l} E \rightarrow \quad \{ \text{push}(\text{newlabel}, \text{true}) \} \\ \quad E_1 \\ \quad \text{and } \{ \text{emit}(\text{'label' top}(\text{true})); \\ \quad \quad \text{pop}(\text{true}) \} \\ \quad E_2 \\ E \rightarrow \text{id} \quad \{ \text{emit}(\text{'if' id.place 'goto' top}(\text{true})); \\ \quad \quad \text{emit}(\text{'goto' top}(\text{false})) \} \end{array}$$

Fig. 5.52. Emitting code for boolean expressions.

Nonoverlapping Lifetimes

A single register is a special case of a stack. If each *push* operation is followed by a *pop*, then there can be at most one element in the stack at a time. In this case, we can use a register instead of a stack. In terms of lifetimes, if the lifetimes of two attributes do not overlap, their values can be held in the same register.

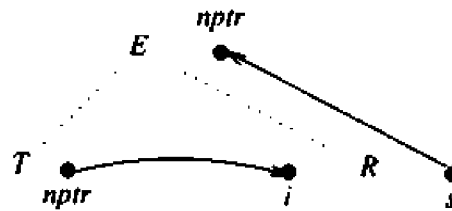
Example 5.28. The syntax-directed definition in Fig. 5.53 constructs syntax trees for list-like expressions with operators at just one precedence level. It is taken from the translation scheme in Fig. 5.28.

PRODUCTION	SEMANTIC RULES
$E \rightarrow T R$	$R.i := T.nptr$ $E.nptr := R.s$
$R \rightarrow \text{addop } T R_1$	$R_1.i := \text{mknode}(\text{addop.lexeme}, R.i, T.nptr)$ $R.s := R_1.s$
$R \rightarrow \epsilon$	$R.s := R.i$
$T \rightarrow \text{num}$	$T.nptr := \text{mkleaf}(\text{num}, \text{num.val})$

Fig. 5.53. A syntax-directed definition adapted from Fig. 5.28.

We claim that the lifetime of each attribute of *R* ends when the attribute that depends on it is evaluated. We can show that, for any parse tree, the *R* attributes can be evaluated into the same register *r*. The following reasoning is typical of that needed to analyze grammars. The induction is on the size of the subtree attached at *R* in the parse tree fragment in Fig. 5.54.

The smallest subtree is obtained if $R \rightarrow \epsilon$ is applied, in which case *R.s* is a copy of *R.i*, so both their values are in register *r*. For a larger subtree, the production at the root of the subtree must be for $R \rightarrow \text{addop } T R_1$. The

Fig. 5.54. Dependency graph for $E \rightarrow TR$.

lifetime of $R.i$ ends when $R_1.i$ is evaluated, so $R_1.i$ can be evaluated into register r . From the inductive hypothesis, all the attributes for instances of nonterminal R in the subtree for R_1 can be assigned the same register. Finally, $R.s$ is a copy of $R_1.s$, so its value is already in r .

The translation scheme in Fig. 5.55 evaluates the attributes in the attribute grammar of Fig. 5.53, using register r to hold the values of the attributes $R.i$ and $R.s$ for all instances of nonterminal R .

```

 $E \rightarrow T$       {  $r := T.nptr$  /*  $r$  now holds  $R.i$  */ }
       $R$       {  $E.nptr := r$  /*  $r$  has returned with  $R.s$  */ }
 $R \rightarrow \text{addop}$ 
       $T$       {  $r := \text{mknode}(\text{addop.lexeme}, r, T.nptr)$  }
       $R$ 
 $R \rightarrow \epsilon$ 
 $T \rightarrow \text{num}$     {  $T.nptr := \text{mkleaf}(\text{num}, \text{num.val})$  }

```

Fig. 5.55. Transformed translation scheme for constructing syntax trees.

For completeness, we show in Fig. 5.56 the code for implementing the translation scheme above; it is constructed according to Algorithm 5.2. Nonterminal R no longer has attributes, so R becomes a procedure rather than a function. Variable r was made local to function E , so it is possible for E to be called recursively, although we do not need to do so in the scheme of Fig. 5.55. This code can be improved further by eliminating tail recursion and then replacing the remaining call of R by the body of the resulting procedure, as in Section 2.5. $\square$

5.10 ANALYSIS OF SYNTAX-DIRECTED DEFINITIONS

In Section 5.7, attributes were evaluated during a traversal of a tree using a set of mutually recursive functions. The function for a nonterminal mapped the values of the inherited attributes at a node to the values of the synthesized attributes at that node.

```

function E: ↑ syntax_tree_node;
    var r: ↑ syntax_tree_node;
        addoplexeme : char;

    procedure R;
    begin
        if lookahead = addop then begin
            addoplexeme := lexval;
            match(addop);
            r := mknode(addoplexeme, r, T);
            R
        end
    end;

begin
    r := T; R
    return r
end;

```

Fig. 5.56. Compare procedure *R* with the code in Fig. 5.31.

The approach of Section 5.7 extends to translations that cannot be performed during a single depth-first traversal. Here, we shall use a separate function for each synthesized attribute of each nonterminal, although groups of synthesized attributes can be evaluated by a single function. The construction in Section 5.7 deals with the special case in which all synthesized attributes form one group. The grouping of attributes is determined from the dependencies set up by the semantic rules in a syntax-directed definition. The following abstract example illustrates the construction of a recursive evaluator.

Example 5.29. The syntax-directed definition in Fig. 5.57 is motivated by a problem we shall consider in Chapter 6. Briefly, the problem is as follows. An “overloaded” identifier can have a set of possible types; as a result, an expression can have a set of possible types. Context information is used to select one of the possible types for each subexpression. The problem can be solved by making a bottom-up pass to synthesize the set of possible types, followed by a top-down pass to narrow down the set to a single type.

The semantic rules in Fig. 5.57 are an abstraction of this problem. Synthesized attribute *s* represents the set of possible types and inherited attribute *i* represents the context information. An additional synthesized attribute *t* that cannot be evaluated in the same pass as *s* might represent the generated code or the type selected for a subexpression. Dependency graphs for the productions in Fig. 5.57 are shown in Fig. 5.58. □

PRODUCTION	SEMANTIC RULES
$S \rightarrow E$	$E.i := g(E.s)$ $S.r := E.t$
$E \rightarrow E_1 E_2$	$E.s := fs(E_1.s, E_2.s)$ $E_1.i := fi1(E.i)$ $E_2.i := fi2(E.i)$ $E.t := ft(E_1.t, E_2.t)$
$E \rightarrow id$	$E.s := id.s$ $E.t := h(E.i)$

Fig. 5.57. Synthesized attributes s and t cannot be evaluated together.

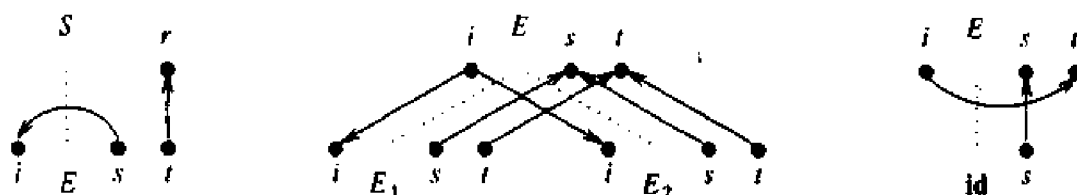


Fig. 5.58. Dependency graphs for productions in Fig. 5.57.

Recursive Evaluation of Attributes

The dependency graph for a parse tree is formed by pasting together smaller graphs corresponding to the semantic rules for a production. The dependency graph D_p for production p is based only on the semantic rules for a single production, i.e., on the semantic rules for the synthesized attributes of the left side and the inherited attributes of the grammar symbols on the right side of the production. That is, the graph D_p shows local dependencies only. For example, all edges in the dependency graph for $E \rightarrow E_1 E_2$ in Fig. 5.58 are between instances of the same attribute. From this dependency graph, we cannot tell that the s attributes must be computed before the other attributes.

A close look at the dependency graph for the parse tree in Fig. 5.59 shows that the attributes of each instance of nonterminal E must be evaluated in the order $E.s$, $E.i$, $E.t$. Note that all attributes in Fig. 5.59 can be evaluated in three passes: a bottom-up pass to evaluate the s attributes, a top-down pass to evaluate the i attributes, and a final bottom-up pass to evaluate the t attributes.

In a recursive evaluator, the function for a synthesized attribute takes the values of some of the inherited attributes as parameters. In general, if synthesized attribute $A.a$ can depend on inherited attribute $A.b$, then the function

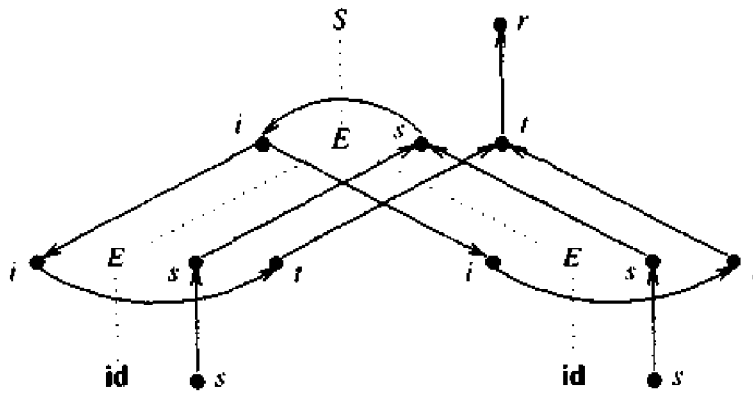


Fig. 5.59. Dependency graph for a parse tree.

for $A.a$ takes $A.b$ as a parameter. Before analyzing dependencies, we consider an example showing their use.

Example 5.30. The functions Es and Et in Fig. 5.60 return the values of the synthesized attributes s and t at a node n labeled E . As in Section 5.7, there is a case for each production in the function for a nonterminal. The code executed in each case simulates the semantic rules associated with the production in Fig. 5.57.

From the above discussion of the dependency graph in Fig. 5.59 we know that attribute $E.t$ at a node in a parse tree may depend on $E.i$. We therefore pass inherited attribute i as a parameter to the function Et for attribute t . Since attribute $E.s$ does not depend on any inherited attributes, function Es has no parameters corresponding to attribute values. $\square$

Strongly Noncircular Syntax-Directed Definitions

Recursive evaluators can be constructed for a class of syntax-directed definitions, called "strongly noncircular" definitions. For a definition in this class, the attributes at each node for a nonterminal can be evaluated according to the same (partial) order. When we construct the function for a synthesized attribute of the nonterminal, this order is used to select the inherited attributes that become the parameters of the function.

We give a definition of this class and show that the syntax-directed definition in Fig. 5.57 falls in the class. We then give an algorithm for testing circularity and strong noncircularity, and show how the implementation of Example 5.30 extends to all strongly noncircular definitions.

Consider nonterminal A at a node n in a parse tree. The dependency graph for the parse tree may in general have paths that start at an attribute of node n , go through attributes of other nodes in the parse tree, and end at another attribute of n . For our purposes, it is enough to look at paths that stay within the part of the parse tree below A . A little thought reveals that such paths go


```

function  $Es(n)$ ;
begin
    case production at node  $n$  of
        ' $E \rightarrow E_1 E_2$ ':
             $s1 := Es(child(n, 1))$ ;
             $s2 := Es(child(n, 2))$ ;
            return  $fs(s1, s2)$ ;
        ' $E \rightarrow id$ ':
            return  $id.s$ ;
        default:
            error
    end
end;

function  $Et(n, i)$ ;
begin
    case production at node  $n$  of
        ' $E \rightarrow E_1 E_2$ ':
             $i1 := fi(i)$ ;
             $t1 := Et(child(n, 1), i1)$ ;
             $i2 := fi2(i)$ ;
             $t2 := Et(child(n, 2), i2)$ ;
            return  $ft(t1, t2)$ ;
        ' $E \rightarrow id$ ':
            return  $h(i)$ ;
        default:
            error
    end
end;

function  $Sr(n)$ ;
begin
     $s := Es(child(n, 1))$ ;
     $i := g(s)$ ;
     $t := Et(child(n, 1), i)$ ;
    return  $t$ 
end;

```

Fig. 5.60. Functions for synthesized attributes in Fig. 5.57.

from some inherited attribute of A to some synthesized attribute of A . We shall make an estimate (possibly too pessimistic) of the set of such paths by considering partial orders on the attributes of A .

Let production p have nonterminals $A_1, A_2, \dots, A_n$ occurring on the right side. Let RA_j be a partial order on the attributes of A_j , for $1 \leq j \leq n$. We

write $D_p[RA_1, RA_2, \dots, RA_n]$ for the graph obtained by adding edges to D_p as follows: if RA_j orders attribute $A_j.b$ before $A_j.c$ then add an edge from $A_j.b$ to $A_j.c$.

A syntax-directed definition is said to be *strongly noncircular* if for each nonterminal A we can find a partial order RA on the attributes of A such that for each production p with left side A and nonterminals $A_1, A_2, \dots, A_n$ occurring on the right side

1. $D_p[RA_1, RA_2, \dots, RA_n]$ is acyclic, and
2. if there is an edge from attribute $A.b$ to $A.c$ in $D_p[RA_1, RA_2, \dots, RA_n]$, then RA orders $A.b$ before $A.c$.

Example 5.31. Let p be the production $E \rightarrow E_1 E_2$ from Fig. 5.57, whose dependency graph D_p is in the center of Fig. 5.58. Let RE be the partial order (total order in this case) $s \rightarrow i \rightarrow t$. There are two occurrences of nonterminals on the right side of p , written E_1 and E_2 , as usual. Thus, RE_1 and RE_2 are the same as RE , and the graph $D_p[RE_1, RE_2]$ is as shown in Fig. 5.61.

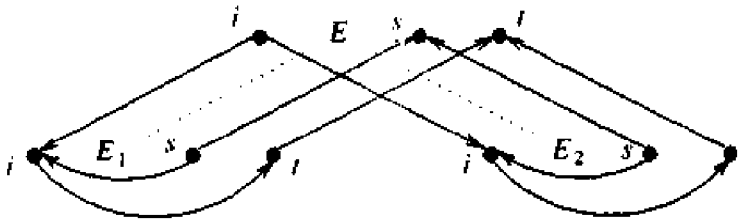


Fig. 5.61. Augmented dependency graph for a production.

Among the attributes associated with the root E in Fig. 5.61, the only paths are from i to t . Since RE makes i precede t , there is no violation of condition (2). $\square$

Given a strongly noncircular definition and partial order RA for each nonterminal A , the function for synthesized attribute s of A takes arguments as follows: if RA orders inherited attribute i before s , then i is an argument of the function, otherwise not.

A Circularity Test

A syntax-directed definition is said to be circular if the dependency graph for some parse tree has a cycle; circular definitions are ill-formed and meaningless. There is no way we can begin to compute any of the attribute values on the cycle. Computing the partial orders that ensure that a definition is strongly noncircular is closely related to testing if a definition is circular. We shall therefore first consider a test for circularity.

Example 5.32. In the following syntax-directed definition, paths between the attributes of A depend on which production is applied. If $A \rightarrow 1$ is applied,

then $A.s$ depends on $A.i$; otherwise, it does not. For complete information about the possible dependencies we therefore have to keep track of sets of partial orders on the attributes of a nonterminal.

PRODUCTION	SEMANTIC RULES
$S \rightarrow A$	$A.i := c$
$A \rightarrow 1$	$A.s := f(A.i)$
$A \rightarrow 2$	$A.s := d$

□

The idea behind the algorithm in Fig. 5.62 is as follows. We represent partial orders by directed acyclic graphs. Given dags on the attributes of symbols on the right side of a production, we can determine a dag for the attributes of the left side as follows.

```
for grammar symbol  $X$  do
     $\mathcal{F}(X)$  has a single graph with the attributes of  $X$  and no edges;
repeat
    change := false;
    for production  $p$  given by  $A \rightarrow X_1X_2 \cdots X_k$  do begin
        for dags  $G_1 \in \mathcal{F}(X_1), \dots, G_k \in \mathcal{F}(X_k)$  do begin
             $D := D_p$ ;
            for edge  $b \rightarrow c$  in  $G_j, 1 \leq j \leq k$  do
                add an edge in  $D$  between attributes  $b$  and  $c$  of  $X_j$ ;
            if  $D$  has a cycle then
                fail the circularity test
            else begin
                 $G :=$  a new graph with nodes for the attributes
                    of  $A$  and no edges;
                for each pair of attributes  $b$  and  $c$  of  $A$  do
                    if there is a path in  $D$  from  $b$  to  $c$  then
                        add  $b \rightarrow c$  to  $G$ ;
                if  $G$  is not already in  $\mathcal{F}(A)$  then begin
                    add  $G$  to  $\mathcal{F}(A)$ ;
                    change := true;
                end
            end
        end
    end
until change = false
```

Fig. 5.62. A circularity test.

Let production p be $A \rightarrow X_1X_2 \cdots X_k$ with dependency graph D_p . Let D_j be a dag for $X_j, 1 \leq j \leq k$. Each edge $b \rightarrow a$ in D_j is temporarily added in to

the dependency graph D_p for the production. If the resulting graph has a cycle, then the syntax-directed definition is circular. Otherwise, paths in the resulting graph determine a new dag on the attributes of the left side of the production, and the resulting dag is added to $\mathcal{F}(A)$.

The circularity test in Fig. 5.62 takes time exponential in the number of graphs in the sets $\mathcal{F}(X)$ for any grammar symbol X . There are syntax-directed definitions that cannot be tested for circularity in polynomial time.

We can convert the algorithm in Fig. 5.62 into a more efficient test if a syntax-directed definition is strongly noncircular, as follows. Instead of maintaining a family of graphs $\mathcal{F}(X)$ for each X , we summarize the information in the family by keeping a single graph $F(X)$. Note that each graph in $\mathcal{F}(X)$ has the same nodes for the attributes of X , but may have different edges. $F(X)$ is the graph on the nodes for the attributes of X that has an edge between $X.b$ and $X.c$ if any graph in $\mathcal{F}(X)$ does. $F(X)$ represents a "worst-case estimate" of dependencies between attributes of X . In particular, if $F(X)$ is acyclic, then the syntax-directed definition is guaranteed to be noncircular. However, the converse need not be true; i.e., if $F(X)$ has a cycle, it is not necessarily true that the syntax-directed definition is circular.

The modified circularity test constructs acyclic graphs $F(X)$ for each X if it succeeds. From these graphs we can construct an evaluator for the syntax-directed definition. The method is a straightforward generalization of Example 5.30. The function for synthesized attribute $X.s$ takes as arguments all and only the inherited attributes that precede s in $F(X)$. The function, called at node n , calls other functions that compute the needed synthesized attributes at the children of n . The routines to compute these attributes are passed values for the inherited attributes they need. The fact that the strong noncircularity test succeeded guarantees that these inherited attributes can be computed.

EXERCISES

- 5.1 For the input expression $(4*7+1)*2$, construct an annotated parse tree according to the syntax-directed definition of Fig. 5.2
- 5.2 Construct the parse tree and syntax tree for the expression $((a)+(b))$ according to
 - a) the syntax-directed definition of Fig. 5.9, and
 - b) the translation scheme of Fig. 5.28.
- 5.3 Construct the dag and identify the value numbers for the subexpressions of the following expression, assuming $+$ associates from the left: $a+a+(a+a+a+(a+a+a))$.
- *5.4 Give a syntax-directed definition to translate infix expressions into infix expressions without redundant parentheses. For example, since $+$ and $*$ associate to the left, $((a*(b+c))*(d))$ can be rewritten as $a*(b+c)*d$.

- 5.5** Give a syntax-directed definition to differentiate expressions formed by applying the arithmetic operators $+$ and $*$ to the variable x and constants; e.g., $x*(3*x + x*x)$. Assume that no simplification takes place, so $3*x$ translates into $3*1 + 0*x$.
- 5.6** The following grammar generates expressions formed by applying an arithmetic operator $+$ to integer and real constants. When two integers are added, the resulting type is integer, otherwise, it is real.

$$E \rightarrow E + T \mid T$$

$$T \rightarrow \text{num} . \text{num} \mid \text{num}$$

- Give a syntax-directed definition to determine the type of each subexpression.
 - Extend the syntax-directed definition of (a) to translate expressions into postfix notation as well as determining types. Use the unary operator `inttoreal` to convert an integer value into an equivalent real value, so that both operands of $+$ in the postfix form have the same type.
- 5.7** Extend the syntax-directed definition of Fig. 5.22 to keep track of the widths of boxes in addition to keeping track of their heights. Assume that terminal `text` has synthesized attribute w giving the normalized width of the text.
- 5.8** Let synthesized attribute *val* give the value of the binary number generated by S in the following grammar. For example, on input 101.101, $S.val = 5.625$

$$S \rightarrow L . L \mid L$$

$$L \rightarrow L B \mid B$$

$$B \rightarrow 0 \mid 1$$

- Use synthesized attributes to determine $S.val$.
 - Determine $S.val$ with a syntax-directed definition in which the only synthesized attribute of B is c , giving the contribution of the bit generated by B to the final value. For example, the contribution of the first and last bits in 101.101 to the value 5.625 is 4 and 0.125, respectively.
- 5.9** Rewrite the underlying grammar in the syntax-directed definition of Example 5.3 so that type information can be propagated using synthesized attributes alone.
- *5.10** When statements generated by the following grammar are translated into abstract machine code, a break statement translates into a jump to the instruction following the nearest enclosing while statement. For simplicity, expressions are represented by the terminal `expr` and other kinds of statements by the terminal `other`. These terminals have a synthesized attribute *code* giving their translation.

$$\begin{array}{l}
 S \rightarrow \text{while expr do begin } S \text{ end} \\
 \quad | \quad S ; S \\
 \quad | \quad \text{break} \\
 \quad | \quad \text{other}
 \end{array}$$

Give a syntax-directed definition translating statements into code for the stack machine of Section 2.8. Make sure that break statements within nested while statements are translated correctly.

- 5.11** Eliminate left recursion from the syntax-directed definitions in Exercise 5.6(a) and (b).
- 5.12** Expressions generated by the following grammar can have assignments within them.

$$\begin{array}{l}
 S \rightarrow E \\
 E \rightarrow E := E \mid E + E \mid (E) \mid \text{id}
 \end{array}$$

The semantics of expressions are as in C. That is, $b := c$ is an expression that assigns the value of c to b ; the r-value of this expression is the same as that of c . Furthermore, $a := (b := c)$ assigns the value of c to b and then to a .

- Construct a syntax-directed definition for checking that the left side of an expression is an l-value. Use an inherited attribute *side* of nonterminal E to indicate whether the expression generated by E appears on the left or right side of an assignment.
 - Extend the syntax-directed definition in (a) to generate intermediate code for the stack machine of Section 2.8 as it checks the input.
- 5.13** Rewrite the underlying grammar of Exercise 5.12 so that it groups the subexpressions of $:=$ to the right and the subexpressions of $+$ to the left.
- Construct a translation scheme that simulates the syntax-directed definition of Exercise 5.12(b).
 - Modify the translation scheme of (a) to emit code incrementally to an output file.
- 5.14** Give a translation scheme for checking that the same identifier does not appear twice in a list of identifiers.
- 5.15** Suppose declarations are generated by the following grammar.

$$\begin{array}{l}
 D \rightarrow \text{id } L \\
 L \rightarrow , \text{id } L \mid : T \\
 T \rightarrow \text{integer} \mid \text{real}
 \end{array}$$

- Construct a translation scheme to enter the type of each identifier into the symbol table, as in Example 5.3.

- b) Construct a predictive translator from the translation scheme in (a).

5.16 The following grammar is an unambiguous version of the underlying grammar in Fig. 5.22. The braces $\{ \}$ are used only for grouping boxes, and are eliminated during translation.

$$\begin{aligned} S &\rightarrow L \\ L &\rightarrow L B \mid B \\ B &\rightarrow B \text{ sub } F \mid F \\ F &\rightarrow \{ L \} \mid \text{text} \end{aligned}$$

- a) Adapt the syntax-directed definition in Fig. 5.22 to use the above grammar.
- b) Convert the syntax-directed definition of (a) into a translation scheme.
- *5.17** Extend the transformation for eliminating left recursion in Section 5.5 to allow the following for nonterminal A in (5.2).
- a) Inherited attributes defined by copy rules.
- b) Inherited attributes.
- 5.18** Eliminate left recursion from the translation scheme of Exercise 5.16(b).
- *5.19** Suppose we have an L-attributed definition whose underlying grammar is either LL(1), or one for which we can resolve ambiguities and construct a predictive parser. Show that we may keep inherited and synthesized attributes on the parser stack of a top-down parser driven by the predictive parsing table.
- *5.20** Prove that adding unique marker nonterminals anywhere in an LL(1) grammar results in a grammar that is LR(1).
- 5.21** Consider the following modification of the LR(1) grammar $L \rightarrow L b \mid a$:
- $$\begin{aligned} L &\rightarrow M L b \mid a \\ M &\rightarrow \epsilon \end{aligned}$$
- a) What order would a bottom-up parser apply the productions in the parse tree for the input string $abbb$?
- *b)** Show that the modified grammar is not LR(1).
- *5.22** Show that in a translation scheme based on Fig. 5.36, the value of inherited attribute $B.ps$ is always immediately below the right side, whenever we reduce a right side to B .
- 5.23** Algorithm 5.3 for bottom-up parsing and translation with inherited attributes uses marker nonterminals to hold the values of inherited attributes at predictable positions in the parser stack. Fewer markers

may be needed if the values are placed on a stack separate from the parsing stack.

- a) Convert the syntax-directed definition in Fig. 5.36 into a translation scheme.
- b) Modify the translation scheme constructed in (a) so that the value of inherited attribute ps appears on a separate stack. Eliminate marker nonterminal M in the process.

***5.24** Consider translation during parsing as in Exercise 5.23. S. C. Johnson suggests the following method for simulating a separate stack for inherited attributes, using markers and a global variable for each inherited attribute. In the following production, the value v is pushed onto stack i by the first action and is popped by the second action:

$$A \rightarrow \alpha \{ \text{push}(v, i) \} \beta \{ \text{pop}(i) \}$$

Stack i can be simulated by the following productions that use a global variable g and a marker nonterminal M with synthesized attribute s :

$$\begin{aligned} A &\rightarrow \alpha M \beta \{ g := M.s \} \\ M &\rightarrow \epsilon \{ M.s := g; g := v \} \end{aligned}$$

- a) Apply this transformation to the translation scheme of Exercise 5.23(b). Replace all references to the top of the separate stack by references to the global variable.
- b) Show that the translation scheme constructed in (a) computes the same values for the synthesized attribute of the start symbol as that in Exercise 5.23(b).

5.25 Use the approach of Section 5.8 to implement all the *E.side* attributes in the translation scheme of Exercise 5.12(b) by a single boolean variable.

5.26 Modify the use of the stack during the depth-first traversal in Example 5.26 so that the values on the stack correspond to those kept on the parser stack in Example 5.19.

BIBLIOGRAPHIC NOTES

The use of synthesized attributes to specify the translation of a language appears in Irons [1961]. The idea of a parser calling for semantic actions is discussed by Samelson and Bauer [1960] and Brooker and Morris [1962]. Along with inherited attributes, dependency graphs and a test for strong non-circularity appear in Knuth [1968] — a test for circularity appears in a correction to the paper. The extended example in the paper uses disciplined side effects to global attributes attached to the root of a parse tree. If attributes can be functions, inherited attributes can be eliminated; as done in denotational semantics, we can associate a function from inherited to synthesized attributes with a nonterminal. Such observations appear in Mayoh [1981].

One application in which side effects in semantic rules are undesirable is syntax-directed editing. Suppose an editor is generated from an attribute grammar for the source language, as in Reps [1984], and consider an editing change to the source program that results in a portion of the parse tree for the program being deleted. As long as there are no side effects, attribute values for the changed program can be recomputed incrementally.

Ershov [1958] uses hashing to keep track of common subexpressions.

The definition of L-attributed grammars in Lewis, Rosenkrantz, and Stearns [1974] is motivated by translation during parsing. Similar restrictions on attribute dependencies apply to each of the left-to-right depth-first traversals in Bochmann [1976]. Affix grammars, as introduced by Koster [1971], are related to L-attributed grammars. Restrictions on L-attributed grammars are proposed in Koskimies and R  ih   [1983] to control access to global attributes.

The mechanical construction of a predictive translator, similar to those constructed by Algorithm 5.2, is described by Bochmann and Ward [1978]. The impression that top-down parsing allows more flexibility for translation is shown to be false by a proof in Brosgol [1974] that a translation scheme based on an $LL(1)$ grammar can be simulated during $LR(1)$ parsing. Independently, Watt [1977] used marker nonterminals to ensure that the values of inherited attributes appear on a stack during bottom-up parsing. Positions on the right sides of productions where marker nonterminals can safely be inserted without losing the $LR(1)$ property are considered in Purdom and Brown [1980] (see Exercise 5.21). Simply requiring inherited attributes to be defined by copy rules is not enough to ensure that attributes can be evaluated during bottom-up parsing; sufficient conditions on semantic rules are given in Tarhio [1982]. A characterization, in terms of parser states, of attributes that can be evaluated during $LR(1)$ parsing, is given by Jones and Madsen [1980]. As an example of a translation that cannot be done during parsing, Giegerich and Wilhelm [1978] consider code generation for boolean expressions. We shall see in Section 8.6 that backpatching can be used for this problem, so a complete second pass is not necessary.

A number of tools for implementing syntax-directed definitions have been developed, starting with FOLDS by Fang [1972], but few have seen widespread use. DELTA by Lorh   [1977] constructed a dependency graph at compile time. It saved space by keeping track of the lifetimes of attributes and eliminating copy rules. Parse-tree based attribute evaluation methods are discussed by Kennedy and Ramanathan [1979] and Cohen and Harry [1979].

Attribute evaluation methods are surveyed by Engelfriet [1984]. A companion paper by Courcelle [1984] surveys the theoretical foundations. HLP, described by R  ih   et al. [1983], makes alternating depth-first traversals, as suggested by Jazayeri and Walter [1975]. LINGUIST by Farrow [1984] also makes alternating passes. Ganzinger et al. [1982] report that MUG allows the order in which children of a node are visited to be determined by the production at the node. GAG, due to Kastens, Hutt, and Zimmerman [1982] allows repeated visits to children of a node. GAG implements the class of ordered

attribute grammars defined by Kastens [1980]. The idea of repeated visits appears in the earlier paper by Kennedy and Warren [1976], where evaluators for the larger class of strongly noncircular grammars are constructed. Saarinen [1978] describes a modification of Kennedy and Warren's method that saves space by keeping attribute values on a stack if they are not needed during a later visit. An implementation described by Jourdan [1984] constructs recursive evaluators for this class. Recursive evaluators are also constructed by Katayama [1984]. A quite different approach is taken in NEATS by Madsen [1980], where a dag is constructed for expressions representing attribute values.

Analysis of dependencies at compiler-construction time can save time and space at compile time. Testing for circularity is a typical analysis problem. Jazayeri, Ogden, and Rounds [1975] prove that a circularity test requires an exponential amount of time as a function of grammar size. Techniques for improving the implementation of a circularity test are considered by Lorho and Pair [1975], R  ih   and Saarinen [1982], and Deransart, Jourdan, and Lorho [1984].

The space used by naive evaluators has led to the development of techniques for conserving space. The algorithm for assigning attribute values to registers in Section 5.8 was described in a quite different context by Marill [1962]. The problem of finding a topological sort of the dependency graph that minimizes the number of registers used is shown to be NP-complete in Sethi [1975]. Compile-time analysis of lifetimes in a multi-pass evaluator appears in R  ih   [1981] and Jazayeri and Pozefsky [1981]. Branquart et al. [1976] mention the use of separate stacks for holding synthesized and inherited attributes during a traversal. GAG performs lifetime analysis and places attribute values in global variables, stacks, and parse tree nodes as needed. A comparison of the space-saving techniques used by GAG and LINGUIST is made by Farrow and Yellin [1984].